

National University of Singapore

LQDDN.Overflower

Nguyen Huu Nhat Quang, Le Pham Duy Khoa, Thieu Nguyen Huy

2026-10-28

Contents

1 Contest 1

2 Mathematics 2

3 Combinatorial 5

4 Number theory 7

5 Numerical 11

6 Data structures 18

7 Graph 29

8 Geometry 41

9 Strings 54

10 Various 58

Contest (1)

template.cpp

15 lines

```
#include <bits/stdc++.h>
#include <cassert>
using namespace std;

#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

.bashrc

3 lines

```
alias c='g++ -Wall -Wconversion -Wfatal-errors -g
    -std=c++17 \
  -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less
    greater' #caps = <>
```

.vimrc

6 lines

```
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark
    ru cul
sy on   |   im jk <esc>   |   im kj <esc>   |
    no ; :
" Select region and then type :Hash to hash your
    selection.
" Useful for verifying that there aren't mistypes
    .
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:
    space:]' \
 \| md5sum \| cut -c-6
```

hash.sh

3 lines

```
# Hashes a file, ignoring all whitespace and
    comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'|
    md5sum |cut -c-6
```

debug.sh

17 lines

```
set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    diff -Z myAnswer correctAnswer > /dev/null ||
        break
    echo "Passed test: "  $i
done
echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
```

pragma.h

2 lines

```
#pragma GCC optimize("O3,Ofast,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
```

troubleshoot.txt

52 lines

```
Pre-submit:
Write a few simple test cases if sample is not
    enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test
    cases?
Can your algorithm handle the whole range of
    input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as
    you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
```

```
Is your output format correct? (including
    whitespace)
Rewrite your solution from the start or let a
    teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of
    any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see
    Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (
    References)
How big is the input and output? (consider scanf)
Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm
    ?

Memory limit exceeded:
What is the max amount of memory your algorithm
    should need?
Are you clearing all data structures between test
     cases?
```

Mathematics (2)

2.1 Math notes

2.1.1 Derangements

Count the number of permutations of length n with 0 fixed points (i.e. $p(i) \neq i$ for all $i \leq n$)

$$f(n) = \begin{cases} 1, & n = 0, \\ 0, & n = 1, \\ (n-1)\big(f(n-1) + f(n-2)\big), & n \geq 2. \end{cases}$$

An equivalent identity is

$$f(n) = n\, f(n-1) + (-1)^n.$$

2.1.2 Catalan Numbers

- Correctly matched parentheses expressions of length $2n$,
- Rooted binary trees with $n+1$ leaves,
- Dyck paths of length $2n$,
- Ways to triangulate a polygon with $n+2$ sides,
- Different binary search trees that can be constructed with n distinct keys.

$$C_n = \frac{1}{n+1}\binom{2n}{n}, \quad n \geq 0.$$

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}, \quad n \geq 0.$$

$$C_0 = 1, \qquad C_{n+1} = \sum_{i=0}^{n} C_i\, C_{n-i}, \quad n \geq 0.$$

The first few values of the sequence (C_n) are

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, ...

2.1.3 Binet's Formula

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{for} \quad n \geq 2.$$

$$F_n = \frac{\phi^n - \psi^n}{\sqrt{5}},$$

where $\phi = \frac{1+\sqrt{5}}{2}$ is the golden ratio and $\psi = \frac{1-\sqrt{5}}{2}$ is its conjugate.

2.1.4 Sums

$$1^4+2^4+3^4+\cdots+n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}.$$

2.1.5 Stirling number of the second kind

The Stirling number of the second kind, $S(n,k)$, counts the number of ways to partition a set of n objects into k non-empty, unlabeled subsets. The formula is recursive and is given by:

$$S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1), \quad S(0,0) = 1, \quad S(n,$$

$$S(n,k) = \frac{1}{k!}\sum_{i=0}^{k}(-1)^{k-i}\binom{k}{i} i^n.$$

2.1.6 Mobius

$$s_{f*g}(n) = \sum_{i=1}^{n}\sum_{d|i} g(d) f(\frac{i}{d})$$

$$s_{f*g}(n) = \sum_{d=1}^{n}\sum_{p=1}^{\lfloor \frac{n}{d} \rfloor} g(d) f(p)$$

$$s_{f*g}(n) = g(1)\sum_{p=1}^{n} f(p) + \sum_{d=2}^{n}\sum_{p=1}^{\lfloor \frac{n}{d} \rfloor} g(d)f(p) = g(1)s_f(n) + \sum_{d=2}^{n} g(d)s_f(\lfloor \frac{n}{d} \rfloor)$$

$$s_f(n) = \frac{s_{f*g}(n) - \sum_{d=2}^{n} s_f(\lfloor \frac{n}{d} \rfloor)g(d)}{g(1)}$$

2.1.7 Combinatorics

$$\sum_{k=0}^{x} \frac{\binom{x}{k}}{\binom{n}{k}} = \frac{n+1}{n-x+1}.$$

2.1.8 Useful Geometry Formulas

Triangle Geometry

Heron's Formula. For a triangle with side lengths a, b, c and semiperimeter

$$s = \frac{a+b+c}{2},$$

the area is

$$A = \sqrt{s(s-a)(s-b)(s-c)}.$$

Area Formulas. For a triangle with side lengths a, b, c, height h_a, circumradius R, and inradius r:

$$A = \frac{1}{2}ah_a, \qquad A = rs, \qquad A = \frac{abc}{4R}, \qquad A = \frac{1}{2}ab\sin\alpha.$$

From these, the circumradius and inradius are:

$$R = \frac{abc}{4A}, \qquad r = \frac{A}{s}.$$

Law of Sines.

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R.$$

Law of Cosines.

$$a^2 = b^2+c^2-2bc\cos A, \quad b^2 = c^2+a^2-2ca\cos B, \quad c^2 = a^2+b^2-2ab\cos C.$$

Analytic Geometry

Let $P(x_0, y_0)$ be a point and a line:

$$\ell : \quad ax + by + c = 0$$

Perpendicular Foot from a Point to a Line.

$$H\left(x_0 - \frac{a(ax_0+by_0+c)}{a^2+b^2}, \quad y_0 - \frac{b(ax_0+by_0+c)}{a^2+b^2}\right).$$

Signed Distance from a Point to a Line.

$$d = \frac{ax_0+by_0+c}{\sqrt{a^2+b^2}}.$$

Reflection of a Point Across a Line.

$$P'\left(x_0 - \frac{2a(ax_0+by_0+c)}{a^2+b^2}, \quad y_0 - \frac{2b(ax_0+by_0+c)}{a^2+b^2}\right).$$

Signed Angle Between Two Vectors. For vectors $\vec{u} = (u_x, u_y)$ and $\vec{v} = (v_x, v_y)$, the signed angle θ from $\vec{u}$ to $\vec{v}$ is

$$\theta = \operatorname{atan2}(u_xv_y - u_yv_x,\ u_xv_x + u_yv_y).$$

Equivalently,

$$\tan\theta = \frac{\vec{u}\times\vec{v}}{\vec{u}\cdot\vec{v}} = \frac{u_xv_y - u_yv_x}{u_xv_x + u_yv_y}.$$

2.2 Algorithms

MillerRabin.h

Description: Primality check for numbers up to $7 \cdot 10^{18}$, extend bases for larger number
Time: $\mathcal{O}(7 \log maxValue)$
Status: Tested on judge.yosupo.jp

fec81f, 36 lines

```
namespace MillerRabin {
    using T = unsigned long long;
    using Int = __uint128_t;

    T binpow(T x, T y, T mod) {
        T ans = 1; x %= mod;
        while (y > 0) {
            if (y & 1) ans = (Int)ans * x % mod;
            x = (Int)x * x % mod, y >>= 1;
        }
        return ans;
    }

    bool notPrime(T n, T base, T d, int s) {
        if (base % n == 0) return false;
        T x = binpow(base % n, d, n);
        if (x == 1 || x == n - 1) return false;
        for (int r = 1; r < s; ++r) {
            x = (Int)x * x % n;
            if (x == n - 1) return false;
        }
        return true;
    }

    bool isPrime(T n) {
        if (n < 2 || n % 6 % 4 != 1) return (n |
            1) == 3;
        int s = __builtin_ctzll(n - 1); T d = n
            >> s;

        const T bases[] = {2, 325, 9375, 28178,
            450775, 9780504, 1795265022};

        for (T base : bases) {
            if (notPrime(n, base, d, s)) return
                false;
```

```
        }
        return true;
    }
}
```

FastFourierTransform.h

Description: FFT to multiply two polynomials.
Usage: `Set MOD. Rounded convolution coefficients must fit in a signed 64-bit integer.`
Time: $\mathcal{O}(N \log N)$ where N is the total sizes of two polynomials
Status: Well-tested

4551f1, 79 lines

```
namespace FFT {
    using ld = long double;
    const int MOD = 1000000007; // Set to the
        required modulus.
    const ld pi = atan2(1, 1) * 4;

    struct cmplx {
        ld a, b;

        cmplx() {
            a = 0, b = 0;
        }

        cmplx(ld a1, ld b1) {
            a = a1, b = b1;
        }
    };

    inline cmplx sum(cmplx& c1, cmplx& c2) {
        return {c1.a + c2.a, c1.b + c2.b};
    }

    inline cmplx sub(cmplx& c1, cmplx& c2) {
        return {c1.a - c2.a, c1.b - c2.b};
    }

    inline cmplx mult(cmplx& c1, cmplx& c2) {
        return {c1.a * c2.a - c1.b * c2.b, c1.a *
             c2.b + c1.b * c2.a};
    }

    void fft(vector<cmplx> &a, int n) {
        for (int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            for (; j >= bit; bit >>= 1) j -= bit;
            j += bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int k = 1; k < n; k *= 2) {
            ld phi = pi / k;
            cmplx wt(cos(phi), sin(phi));
            for (int i = 0; i < n; i += 2 * k) {
                cmplx w(1, 0);
                for (int j = 0; j < k; ++j) {
                    cmplx u = a[i + j], v = mult(
                        a[i + j + k], w);
                    a[i + j] = sum(u, v);
                    a[i + j + k] = sub(u, v);
                    w = mult(w, wt);
                }
            }
        }
    }

    vector<int> multiply(vector<int> &s, vector<
        int> &t) {
        if (s.empty() || t.empty()) return {};
        int need = (int) s.size() + t.size() - 1,
             sz = 1;
        while (sz < need) sz <<= 1;
        vector<cmplx> a(sz), b(sz);
        for (int i = 0; i < sz; ++i) {
            if(i < (int)s.size()) a[i] = cmplx(s[
                i], 0);
            else a[i] = cmplx(0, 0);
            if(i < (int)t.size()) b[i] = cmplx(t[
                i], 0);
            else b[i] = cmplx(0, 0);
        }
        fft(a, sz);
        fft(b, sz);
        for (int i = 0; i < sz; ++i) {
            a[i] = mult(a[i], b[i]);
        }
        fft(a, sz);
        reverse(a.begin() + 1, a.end());

        vector<int> ans(need);
        for (int i = 0; i < need; ++i) ans[i] = (
            llround(a[i].a / sz) % MOD + MOD) %
            MOD;
        return ans;
    }
}
vector<int> operator * (vector<int> &s, vector<
    int> &t) {
    return FFT::multiply(s, t);
}
```

GaussElimination.h

Description: Gauss elimation to solve system of linear equations
Usage: `variables are numbered 0 -> m-1, the m-th column is the right hand side`
Time: $\mathcal{O}(min(N, M) \cdot N \cdot M)$ where N = equations, M = variables
Status: Tested on VNOI

9990b6, 38 lines

```
int gauss(vector<vector<double>> &a, vector<
    double> &ans) {
    const double EPS = 1e-9;
    int n = a.size(), m = a[0].size() - 1;

    vector<int> where(m, -1);
    for (int col = 0, row = 0; col < m && row < n
        ; ++col) {
        int p = row;
        for (int i = row; i < n; ++i)
            if (abs(a[i][col]) > abs(a[p][col]))
                p = i;

        if (abs(a[p][col]) < EPS) continue;

        swap(a[row], a[p]);
        where[col] = row;

        for (int i = 0; i < n; ++i) if (i != row)
             {
            double c = a[i][col] / a[row][col];
```

```
            for (int j = col; j <= m; ++j)
                a[i][j] -= a[row][j] * c;
        }
        ++row;
    }

    ans.assign(m, 0);
    for (int i = 0; i < m; ++i) if (where[i] !=
        -1) {
        ans[i] = a[where[i]][m] / a[where[i]][i];
    }

    for (int i = 0; i < n; ++i) {
        double sum = 0;
        for (int j = 0; j < m; ++j) sum += ans[j]
             * a[i][j];
        if (abs(sum - a[i][m]) > EPS) return 0;
    }

    for (int i = 0; i < m; ++i)
        if (where[i] == -1) return 2;
    return 1;
}
```

GaussElimination2.h

Description: Gauss elimation to solve system of linear equations (mod 2)

Usage: `all roots of the system have the form: ans + sum(c(i) * basis(i))`

Time: $\mathcal{O}\left(\frac{min(N,M)\cdot N\cdot M}{64}\right)$ where N = equations, M = variables

Status: Tested on judge.yosupo.jp

1a50a3, 55 lines

```
namespace Gauss {
    // n: num equations, m: num variables
    // left hand side: [0, m-1], right hand side:
         column m
    const int M = 4096 + 5;
    using T = bitset<M>;

    int n, m;
    int where[M];
    vector<T> a;
    T ans;

    void init(int _n, int _m, const vector<T> &_a
        ) {
        n = _n, m = _m, a = _a;
    }

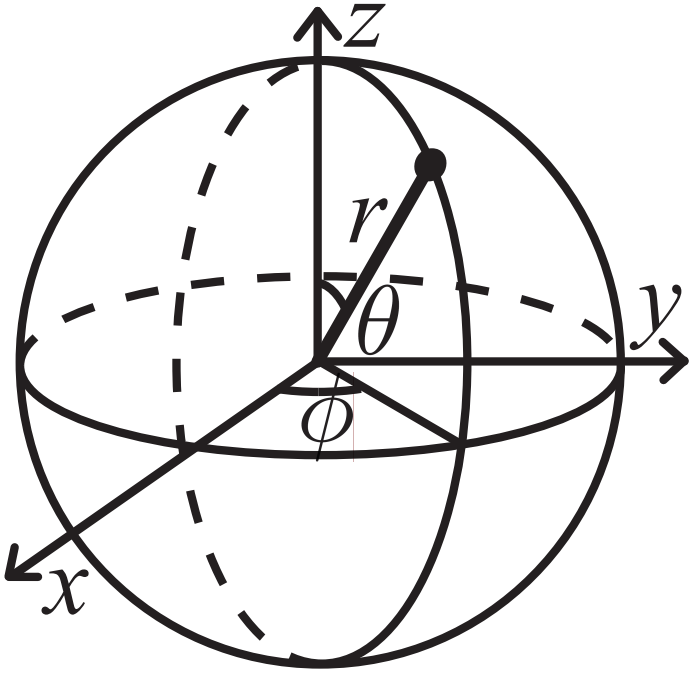
    int gauss() {
        ans.reset();
        for (int i = 0; i < m; ++i) where[i] =
            -1;
        int row = 0;
        for (int col = 0; col < m && row < n; ++
            col) {
            for (int i = row; i < n; ++i) if (a[i
                ][col]) {
                swap(a[i], a[row]);
                break;
            }
            if (!a[row][col]) continue;
            where[col] = row;
            for (int i = 0; i < n; ++i)
                if (i != row && a[i][col]) a[i] ^
                    = a[row];
            ++row;
        }

        for (int i = 0; i < m; ++i) if (where[i]
            != -1) {
            ans[i] = a[where[i]][m];
        }

        for (int i = 0; i < n; ++i) {
            bool sum = 0;
            for (int j = 0; j < m; ++j) sum ^=
                ans[j] & a[i][j];
            if (sum != a[i][m]) return -1;
        }
        return row;
    }

    vector<T> get_basis() { // assume gauss() has
         been called
        vector<T> basis;
        for (int i = 0; i < m; ++i) if (where[i]
            == -1) {
            T x; x[i] = 1;
            // x(j) + a[where[j]][i] * x[i] = 0
            for (int j = 0; j < m; ++j)
                if (where[j] != -1) x[j] = a[
                    where[j]][i];
            basis.emplace_back(x);
        }
        return basis;
    }
}
```

2.2.1 sphericalCoordinates



Combinatorial (3)

3.1 Permutations

3.1.1 Factorial

n	1	2	3	4	5	6	7	8	9	10
$n!$	1	2	6	24	120	720	5040	40320	362880	3628800
n	11	12	13	14	15	16	17			
$n!$	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
n	20	25	30	40	50	100	150	171		
$n!$	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h

Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.

Time: $\mathcal{O}(n)$

044568, 6 lines

```
int permToInt(vi& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(
      use & -(1<<x)),
    use |= 1 << x;                    // (note:
        minus, not ~!)
  return r;
}
```

3.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

3.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

3.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

3.2 Partitions and subsets

3.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

3.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

3.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!...k_n!}$.

Status: Tested on kattis:lexicography

a0a312, 5 lines

```
ll multinomial(vi& v) {
  ll c = 1, m = v.empty() ? 1 : v[0];
  rep(i,1,sz(v)) rep(j,0,v[i]) c = c * ++m / (j
      +1);
  return c;
}
```

3.3 General purpose numbers

3.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

3.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$
$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$

$c(8,k) =$
$0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) =$
$0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

3.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

3.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

3.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

3.3.6 Labeled unrooted trees

\# on n vertices: n^{n-2}
\# on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
\# with degrees d_i:
$(n-2)!/((d_1-1)! \cdots (d_n-1)!)$

3.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2} C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Number theory (4)

4.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set `mod` to some number first and then you can use the structure.

"euclid.h" 35bfea, 18 lines

```
const ll mod = 17; // change to something else
struct Mod {
  ll x;
  Mod(ll xx) : x(xx) {}
  Mod operator+(Mod b) { return Mod((x + b.x) %
      mod); }
  Mod operator-(Mod b) { return Mod((x - b.x +
      mod) % mod); }
  Mod operator*(Mod b) { return Mod((x * b.x) %
      mod); }
  Mod operator/(Mod b) { return *this * invert(b)
      ; }
  Mod invert(Mod a) {
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1); return Mod((x + mod) % mod);
  }
  Mod operator^(ll e) {
    if (!e) return Mod(1);
    Mod r = *this ^ (e / 2); r = r * r;
    return e&1 ? *this * r : r;
  }
};
```

ModInverse.h

Description: Pre-computation of modular inverses. Assumes LIM $\leq$ mod and that mod is a prime.

Status: Works

db454d, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
vector<ll> inv(LIM); inv[1] = 1;
rep(i,2,LIM) inv[i] = mod - (mod / i) * inv[mod %
    i] % mod;
```

ModPow.h

Status: tested

b83e45, 7 lines

```
const ll mod = 1000000007; // faster if const
ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}(\sqrt{m})$
Status: tested for all 0 <= a,x < 500 and 0 < m < 500.
c040b8, 11 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
modsum(to, c, k, m) $= \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.
Time: $\log(m)$, with a large constant.
Status: Tested for all —k—,—c—,to,m <= 50, and on kattis:aladin
5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1);
    }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 -
      c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to,
      c, k, m);
}
```

ModMul.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for modmul, $\mathcal{O}(\log b)$ for modpow
Status: stress-tested, proven correct
446484, 33 lines

```
typedef unsigned int uint;
typedef unsigned long long ull;
uint modmul(uint a, uint b, uint M) {
#ifdef _WIN32
  ull x = ull(a) * ull(b);
  uint xh = uint(x >> 32), xl = uint(x), d, m;
  asm("divl %4; \n\t" : "=a"(d), "=d"(m)
    : "d"(xh), "a"(xl), "r"(M()));
  return m;
#else
  uint ret;
  asm("mul %%ebx\ndiv %%ecx\nmov %%edx, %0\n"
  : "=ret"(ret): "a"(a), "b"(b), "c"(M): "edx");
  return ret;
#endif
}
ull modmul(ull a, ull b, ull M) {
#ifdef _WIN32
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M)
      ;
#else
  ull ret;
  asm("mul %%rbx\ndiv %%rcx\n" : "=d"(ret)
    : "a"(a), "b"(b), "c"(M));
  return ret;
#endif
}
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p
Status: Tested for all a,p <= 10000
19dd12, 34 lines

```
namespace mod_sqrt_detail {
ll power(ll a, ll e, ll p) {
    ll res = 1;
    for (; e; e >>= 1, a = a * a % p)
        if (e & 1) res = res * a % p;
    return res;
}
}

ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
    if (p == 2) return a;
  if (a == 0) return 0;
  assert(mod_sqrt_detail::power(a, (p-1)/2, p) ==
      1); // else no solution
  if (p % 4 == 3) return mod_sqrt_detail::power(a
      , (p+1)/4, p);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if
      p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (mod_sqrt_detail::power(n, (p - 1) / 2, p
      ) != p - 1) ++n;
  ll x = mod_sqrt_detail::power(a, (s + 1) / 2, p
      );
  ll b = mod_sqrt_detail::power(a, s, p), g =
      mod_sqrt_detail::power(n, s, p);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = mod_sqrt_detail::power(g, 1LL << (r -
        m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
```

```
    b = b * g % p;
  }
}
```

4.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s
Status: Stress-tested

6b2912, 20 lines

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM /
      2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log
      (LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i])
      {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve
        [j] = 1;
  }
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p))
          block[i-L] = 1;
    rep(i,0,min(S, R - L))
      if (!block[i]) pr.push_back((L + i) * 2 +
          1);
  }
  for (int i : pr) isPrime[i] = 1;
  return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.
Status: Stress-tested

"ModMul.h" 60dcd1, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) ==
      3;
  ull A[] = {2, 325, 9375, 28178, 450775,
      9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.
Status: stress-tested

"ModMul.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) +
      i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))
        ) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

4.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
}
```

Euclid.java

Description: Finds {x, y, d} s.t. ax + by = d = gcd(a, b).
Status: Tested

6aba01, 11 lines

```
static BigInteger[] euclid(BigInteger a,
    BigInteger b) {
  BigInteger x = BigInteger.ONE, yy = x;
  BigInteger y = BigInteger.ZERO, xx = y;
  while (b.signum() != 0) {
    BigInteger q = a.divide(b), t = b;
    b = a.mod(b); a = t;
    t = xx; xx = x.subtract(q.multiply(xx)); x =
        t;
    t = yy; yy = y.subtract(q.multiply(yy)); y =
        t;
  }
  return new BigInteger[]{x, y, a};
}
```

CRT.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$
Status: Works

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
```

```
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

4.3.1 Bézout's identity

For $a \neq 0$, $b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} ... (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1 \leq k \leq n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod{n}$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod{p}$ $\forall a$.
Status: Tested

cf7d6d, 8 lines

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi() {
  rep(i,0,LIM) phi[i] = i&1 ? i : i/2;
  for (int i = 3; i < LIM; i += 2) if(phi[i] == i
      )
    for (int j = i; j < LIM; j += i) phi[j] -=
        phi[j] / i;
}
```

4.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. ($p_k/q_k$ alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞; if x is the root of a degree 2 polynomial the a's eventually become cyclic.
Time: $\mathcal{O}(\log N)$
Status: stress-tested for n <= 300

dd6c5e, 21 lines

```
typedef double d; // for N ~ 1e7; long double for
    N ~ 1e9
pair<ll, ll> approximate(d x, ll N) {
  ll LP = 0, LQ = 1, P = 1, Q = 0, inf =
      LLONG_MAX; d y = x;
  for (;;) {
    ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ)
        / Q : inf),
      a = (ll)floor(y), b = min(a, lim),
      NP = b*P + LP, NQ = b*Q + LQ;
    if (a > b) {
      // If b > a/2, we have a semi-convergent
          that gives us a
      // better approximation; if b = a/2, we *
          may* have one.
      // Return {P, Q} here for a more canonical
          approximation.
      return (abs(x - (d)NP / (d)NQ) < abs(x - (d
          )P / (d)Q)) ?
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if (abs(y = 1/(y - (d)a)) > 3*N) {
      return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
  }
}
```

FracBinarySearch.h

Description: Given f and N, finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: `fracBS([](Frac f) { return f.p>=3*f.q; }, 10); // {1,3}`
Time: $\mathcal{O}(\log(N))$
Status: stress-tested for n <= 300

27ab3e, 25 lines

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N) {
  bool dir = 1, A = 1, B = 1;
  Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to
      search (0, N]
  if (f(lo)) return lo;
  assert(f(hi));
  while (A || B) {
    ll adv = 0, step = 1; // move hi if dir, else
        lo
    for (int si = 0; step; (step *= 2) >>= si) {
      adv += step;
      Frac mid{lo.p * adv + hi.p, lo.q * adv + hi
          .q};
      if (abs(mid.p) > N || mid.q > N || dir == !
          f(mid)) {
        adv -= step; si = 2;
      }
    }
    hi.p += lo.p * adv;
    hi.q += lo.q * adv;
    dir = !dir;
    swap(lo, hi);
    A = B; B = !!adv;
  }
  return dir ? hi : lo;
}
```

4.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

4.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

4.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

4.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Eratosthenes.h

Description: Prime sieve for generating all primes up to a certain limit. isprime[i] is true iff i is a prime.
Time: lim=100'000'000 $\approx$ 0.8 s. Runs 30% faster if only odd indices are stored.
Status: Tested

7c144c, 11 lines

```
const int MAX_PR = 5'000'000;
bitset<MAX_PR> isprime;
vi eratosthenesSieve(int lim) {
  isprime.set(); isprime[0] = isprime[1] = 0;
  for (int i = 4; i < lim; i += 2) isprime[i] =
      0;
  for (int i = 3; i*i < lim; i += 2) if (isprime[
      i])
    for (int j = i*i; j < lim; j += i*2) isprime[
        j] = 0;
  vi pr;
  rep(i,2,lim) if (isprime[i]) pr.push_back(i);
  return pr;
}
```

Numerical (5)

5.1 Polynomials and recurrences

Polynomial.h

c9b7b0, 17 lines

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  }
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  }
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i
        +1]*x0+b, b=c;
    a.pop_back();
  }
};
```

PolyRoots.h

Description: Finds the real roots to a polynomial.
Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin,
    double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
Time: $\mathcal{O}\left(n^2\right)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}\left(N^2\right)$
Status: bruteforce-tested mod 5 for n <= 5 and all s

"../number-theory/ModPow.h" 96548b, 20 lines

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1;

  ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) %
        mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  }

  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
}
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots \geq n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}\left(n^2 \log k\right)$
Status: bruteforce-tested mod 5 for n <= 5

f4e444, 26 lines

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);

  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) %
          mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] *
          tr[j]) % mod;
    res.resize(n + 1);
    return res;
  };

  Poly pol(n + 1), e(pol);
  pol[0] = e[1] = 1;

  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  }
  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) %
      mod;
  return res;
}
```

5.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in the Various chapter for a discrete version.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$
Status: tested

31d45b, 14 lines

```
double gss(double a, double b, double (*f)(double
    )) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else {
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    }
  return a;
}
```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.
Status: used with great success

8eeeaf, 14 lines

```
typedef array<double, 2> P;
```

```
template<class F> pair<double, P> hillClimb(P
    start, F f) {
  pair<double, P> cur(f(start), start);
  for (double jmp = 1e9; jmp > 1e-20; jmp /= 2) {
    rep(j,0,100) rep(dx,-1,2) rep(dy,-1,2) {
      P p = cur.second;
      p[0] += dx*jmp;
      p[1] += dy*jmp;
      cur = min(cur, make_pair(f(p), p));
    }
  }
  return cur;
}
```

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.
Status: mostly untested

4756fc, 7 lines

```
template<class F>
double quad(double a, double b, F f, const int n
    = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2)
    v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
}
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.
Usage:

```
double sphereVolume = quad(-1, 1,
[](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});
```

Status: mostly untested

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (
    b-a) / 6

template <class F>
d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b,
      eps / 2, S2);
}
template<class F>
d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
}
```

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage:

```
vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
```

Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.
Status: seems to work?

aa8530, 68 lines

```
typedef double T; // long double, Rational,
    double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X
    [s],N[s])) s=j

struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c
      ) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n
        +2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i
          ][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps
        ) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[
            i])
                    < MP(D[r][n+1] / D[r][s], B[
                        r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }
```

```
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps)
          return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

5.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}\left(N^3\right)$
Status: somewhat tested

bd5cec, 15 lines

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i
        ])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i
          ][k];
    }
  }
  return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3\right)$
Status: bruteforce-tested for N <= 3, mod <= 7

3313dc, 18 lines

```
const ll mod = 12345;
ll det(vector<vector<ll>>& a) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod
              ;
        swap(a[i], a[j]);
        ans *= -1;
      }
    }
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  }
  return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}\left(n^2m\right)$
Status: tested on kattis:equationsolver, and bruteforce-tested mod 3 and 5 for n,m <= 3

44c9ab, 38 lines

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m
      )
}
```

SolveLinear2.h

Description: To get all uniquely determined values of x back from SolveLinear, make the following changes:
Status: tested on kattis:equationsolverplus, stress-tested

"SolveLinear.h" 08e495, 7 lines

```
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto
      fail;
```

```
  x[col[i]] = b[i] / A[i][i];
fail:; }
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Time: $\mathcal{O}\left(n^2 m\right)$
Status: bruteforce-tested for n, m <= 4

fa2d7a, 34 lines

```
typedef bitset<1000> bs;

int solveLinear(vector<bs>& A, vi& b, bs& x, int
    m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break
        ;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    }
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    }
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    }
    rank++;
  }

  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  }
  return rank; // (multiple solutions if rank < m
      )
}
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$
Status: Slightly tested

ebfff6, 35 lines

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n)
      );
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp
          [j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    }
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  }

  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i
      ][j];
  return n;
}
```

MatrixInverse-mod.h

Description: Invert matrix A modulo a prime. Returns rank; result is stored in A unless singular (rank < n). For prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1})$ (mod p^k) where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$
Status: Slightly tested

"../number-theory/ModPow.h" 0b7b13, 37 lines

```
int matInv(vector<vector<ll>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<ll>> tmp(n, vector<ll>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;

  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n) if (A[j][k]) {
      r = j; c = k; goto found;
    }
    return i;
found:
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp
          [j][c]);
    swap(col[i], col[c]);
    ll v = modpow(A[i][i], mod - 2);
    rep(j,i+1,n) {
      ll f = A[j][i] * v % mod;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] = (A[j][k] - f*A[i][k
          ]) % mod;
      rep(k,0,n) tmp[j][k] = (tmp[j][k] - f*tmp[i
          ][k]) % mod;
    }
    rep(j,i+1,n) A[i][j] = A[i][j] * v % mod;
```

```
    rep(j,0,n) tmp[i][j] = tmp[i][j] * v % mod;
    A[i][i] = 1;
  }

  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    ll v = A[j][i];
    rep(k,0,n) tmp[j][k] = (tmp[j][k] - v*tmp[i][
        k]) % mod;
  }

  rep(i,0,n) rep(j,0,n)
    A[col[i]][col[j]] = tmp[i][j] % mod + (tmp[i
        ][j] < 0)*mod;
  return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$
Status: Brute-force tested mod 5 and 7 and stress-tested for real matrices obeying the criteria above.

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const
    vector<T>& super,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { //
        diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] /
          super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else {
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    }
  }
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else {
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    }
  }
  return b;
}
```

5.4 Fourier transforms

FastFourierTransform.h

Description: fft(a) computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.
Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)
Status: somewhat tested

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1);
  static vector<C> rt(2, 1);  // (^ 10% faster if
      double)
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x
        : R[i/2];
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L)
      / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i
      ]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k)
        {
      C z = rt[j+k] * a[i+j+k]; // (25% faster if
          hand-rolled)
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    }
}
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L
      ;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[
      i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n
      );
  return res;
}
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.
Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)
Status: stress-tested

"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl
    &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=
      int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a
      [i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b
      [i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 *
        n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 *
        n) / 1i;
  }
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(
        outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i
        ])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv)
        % M;
  }
  return res;
}
```

NumberTheoreticTransform.h

Description: ntt(a) computes $\hat{f}(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. `conv(a, b) = c`, where $c[x] = \sum a[i] b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in [0, mod).
Time: $\mathcal{O}(N \log N)$
Status: stress-tested

ccea62, 41 lines

```
const ll mod = (119 << 23) + 1, root = 62; // =
    998244353
ll nttPow(ll a, ll e) {
    ll res = 1;
    for (; e; e >>= 1, a = a * a % mod)
        if (e & 1) res = res * a % mod;
    return res;
}
// For p < 2^30 there is also e.g. 5 << 25, 7 <<
    26, 479 << 21
// and 483 << 21 (same root). The last two are >
    10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s
      ++) {
    rt.resize(n);
    ll z[] = {1, nttPow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] %
        mod;
  }
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L)
      / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i
      ]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k)
        {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai
          = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    }
}
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 -
      __builtin_clz(s),
      n = 1 << B;
  int inv = nttPow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod *
        inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.
Time: $\mathcal{O}(N \log N)$
Status: stress-tested

464cf3, 16 lines

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *=
      2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i
        ,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v)
          =
        inv ? pii(v - u, u) : pii(v, u + v); //
            AND
        inv ? pii(v, u - v) : pii(u + v, u); //
            OR
        pii(u + v, u - v);                   //
            XOR
    }
```

```
  }
  if (inv) for (int& x : a) x /= sz(a); // XOR only
}
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
}
```

Data structures (6)

OrderSet.h

Description: Ordered set.
Usage: 0-indexed.
find_by_order(x) returns x-th element (ascending order)
order_of_key(x) returns no. of elements <x.
Time: $\mathcal{O}(\log N)$ for each operation
Status: Tested on one problem.

<ext/pb_ds/assoc_container.hpp> df383f, 3 lines

```
using namespace __gnu_pbds;
template <class T>
using orderset = tree<T, null_type, less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
```

FastSet.h

Description: Fast set for sets that only have integers in [1, N]
Usage: call init() first
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(log64(n))$ for update and lowerbound, upperbound
Status: Well-tested

e52539, 92 lines

```
struct FastSet {
    static const int N = 250000 + 5;
    static const int BLOCK = 64;
    static const int LAYER = 3;
    uint64_t a[N + 5]{};
    int layer_start[LAYER];
    FastSet(){
        int start = 0, pos = 1;
        for(int i = 0; i < LAYER; ++i) {
            layer_start[i] = start;
            start += pos;
            pos = pos * BLOCK;
        }
    }
    void init() { fill(a, a + N + 5, 0); }

    void update(int x) {
        assert(1 <= x && x <= N);
        //add if x not in set and vice versa
        int id = x >> 6, prev = layer_start[LAYER
            - 1] + id;
        a[prev] ^= (1ULL << (x & (BLOCK - 1)));
        for (int i = LAYER - 2; i >= 0; --i) {
            x >>= 6, id >>= 6;
            int digit = x & (BLOCK - 1), cur = id
                + layer_start[i];
            if ((a[prev] > 0) != ((a[cur] >>
                digit) & 1ULL)) a[cur] ^= (1ULL
                << digit);
            prev = cur;
        }
    }

    int get_next(uint64_t mask, int l) {
        if (l >= BLOCK) return -1;
        mask &= ~uint64_t(0) << l;
        return mask ? __builtin_ctzll(mask) : -1;
    }

    int find_next(int x) { // return first ele >=
        x or -1 if no ele
        int id = x >> 6;
        int foo = get_next(a[id + layer_start[
            LAYER - 1]], x & (BLOCK - 1));
        if (foo != -1) return x - (x & (BLOCK -
            1)) + foo;
        x >>= 6, id >>= 6;
        for (int i = LAYER - 2; i>=0; --i) {
            int digit = (x & (BLOCK - 1)) + 1;
            int cur = layer_start[i] + id;
            int foo = get_next(a[cur], digit);
            if (foo != -1) {
                id = (id << 6) + foo;
                for (int j = i + 1; j < LAYER; ++
                    j) {
                    int digit = __builtin_ctzll(a
                        [id + layer_start[j]]);
                    id = (id << 6) + digit;
                }
                return id;
            }
            x >>= 6, id >>= 6;
        }
        return -1;
    }

    int get_prev(uint64_t mask, int l) {
        if (l < BLOCK) mask &= (1ULL << l) - 1;
        return mask ? 63 - __builtin_clzll(mask)
            : -1;
    }

    int find_prev(int x) { // return last ele < x
        or -1 if no ele
        int id = x >> 6;
        int foo = get_prev(a[id + layer_start[
            LAYER - 1]], x & (BLOCK - 1));
        if (foo != -1) return x - (x & (BLOCK -
            1)) + foo;
        x >>= 6, id >>= 6;
        for(int i = LAYER - 2; i>=0; --i) {
            int digit = (x & (BLOCK - 1)) + 1;
            int cur = layer_start[i] + id;
            int foo = get_prev(a[cur], digit - 1)
                ;
            if (foo != -1) {
                id = (id << 6) + foo;
                for(int j = i + 1; j < LAYER; ++j
                    ) {
                    int digit = (63 -
                        __builtin_clzll(a[id +
                        layer_start[j]]));
                    id = (id << 6) + digit;
                }
                return id;
```

```
            }
            x >>= 6, id >>= 6;
        }
        return -1;
    }

    int get_min() {
        return find_next(1);
    }

    int get_max() {
        return find_prev(N + 1);
    }
};
```

LichaoTree.h

Description: Li Chao Tree, can query min and add line.
Time: Both operations are $\mathcal{O}(\log N)$
Status: Tested by KickingKun

764eb9, 49 lines

```
struct LiChaoTree {
    //range for x is in [-lim, lim].
    static const ll lim = 1e9, INF = 1e18;
    struct Line {
        ll a, b;
        ll operator() (ll x) {
            return a * x + b;
        }
    };

    struct node {
        Line line;
        node *l, *r;
        node() {
            line = {0, INF};
            l = r = NULL;
        }
    };

    node *root = new node();

    ll query(node *i, ll l, ll r, ll x) {
        if (i == NULL || x < l || x > r) return
            INF;
        ll m = (l + r) >> 1;
        ll ans = i->line(x);
        ans = min(ans, query(i->l, l, m, x));
        ans = min(ans, query(i->r, m + 1, r, x));
        return ans;
    }

    ll query(ll x) {
        return query(root, -lim, lim, x);
    }

    void add(Line li, node *&cur, ll l, ll r) {
        if (cur == nullptr){
            cur = new node(); cur->line = li;
            return;
        }
        ll mid = (l + r) >> 1;
        if (li(mid) < cur->line(mid)) swap(li,
            cur->line);
        if (li(l) < cur->line(l)) add(li, cur->l,
             l, mid);
        if (li(r) < cur->line(r)) add(li, cur->r,
             mid + 1, r);
    }

    void add(ll m, ll b) {
        add({m, b}, root, -lim, lim);
    }
} cht;
```

PersistentIT.h

Description: Persistent segment tree with point add updates and range sum queries.
Usage: Remember to call mytree.resize(n) before any operations.
Make sure to check for integer overflows in update values.
Memory: $\mathcal{O}(\log N \cdot numUpdates)$
Time: $\mathcal{O}(\log N)$ for each operation
Status: Tested on CSES

c22699, 53 lines

```
//change the parameters
const int MAX_UPDATES = 1e7+10;
struct Node {
    int le, ri;
    ll sum;
    Node(int _le = 0, int _ri = 0, ll _sum = 0) {
        le = _le, ri = _ri, sum = _sum;
    }
} nodes[MAX_UPDATES];
struct PersistentIT { // 1-indexed array
    int n;
    PersistentIT(int _n = 0) {
        resize(_n);
    }

    void resize(int _n) {
        n = _n;
    }
    int numNode = 0;
    int update(int oldId, int l, int r, int p,
        int val) {
        if (l > p || r < p) return oldId;

        assert(numNode + 1 < MAX_UPDATES);
        numNode++; int id = numNode;
        if (l == r) {
            nodes[id] = nodes[oldId];
            nodes[id].sum += val;
            return id;
        }

        int mid = (l + r) >> 1;
        nodes[id].le = update(nodes[oldId].le, l,
             mid, p, val);
        nodes[id].ri = update(nodes[oldId].ri,
            mid + 1, r, p, val);

        nodes[id].sum = nodes[nodes[id].le].sum +
             nodes[nodes[id].ri].sum;
        return id;
    }

    int update(int root, int p, int val) {
        return update(root, 1, n, p, val);
    }
```

```
    ll get(int id, int l, int r, int u, int v) {
        if (l > v || r < u) return 0;
        if (u <= l && r <= v) return nodes[id].
            sum;
        int mid = (l + r) >> 1;
        return get(nodes[id].le, l, mid, u, v) +
            get(nodes[id].ri, mid + 1, r, u, v);
    }

    ll get(int root, int u, int v) {
        return get(root, 1, n, u, v);
    }
} mytree;
```

PersistentLazyIT.h

Description: Persistent segment tree with range updates and range sum queries.
Usage: Remember to call mytree.resize(n) before any operations.
Memory: $\mathcal{O}(\log N \cdot numUpdates)$
Time: $\mathcal{O}(\log N)$ amortized per update, $\mathcal{O}(\log N)$ per query
Status: Randomized version-history regression test

e5c3a6, 78 lines

```
struct Node {
    int le, ri;
    ll lazy, sum;

    Node(int _le = 0, int _ri = 0, ll _lazy = 0,
        ll _sum = 0) {
        le = _le, ri = _ri, lazy = _lazy, sum =
            _sum;
    }
};
vector<Node> nodes(1); // Index 0 is the all-zero
     subtree.
int numNode = 0;
vector<int> version;

struct PersistentLazyIT {
    int n;

    PersistentLazyIT(int _n = 0) {
        resize(_n);
    }

    void resize(int _n) {
        n = _n;
    }

    int newlazykid(int oldId, int l, int r, ll
        val) {
        int id = ++numNode;
        nodes.push_back(nodes[oldId]);

        nodes[id].sum += 1ll * val * (r - l + 1);
        nodes[id].lazy += val;

        return id;
    }

    int newparent(int le, int ri) {
        int id = ++numNode;
        nodes.emplace_back();
        nodes[id].le = le, nodes[id].ri = ri;
        nodes[id].sum = nodes[le].sum + nodes[ri
            ].sum;
        return id;
    }

    void push(int id, int l, int r) {
        if (nodes[id].lazy == 0) return;
        if (l != r) {
            int mid = (l + r) >> 1;
            nodes[id].le = newlazykid(nodes[id].
                le, l, mid, nodes[id].lazy);
            nodes[id].ri = newlazykid(nodes[id].
                ri, mid + 1, r, nodes[id].lazy);
        }
        nodes[id].lazy = 0;
    }

    int update(int oldId, int l, int r, int u,
        int v, ll val) {
        if (l > v || r < u) return oldId;
        if (u <= l && r <= v) return newlazykid(
            oldId, l, r, val);
        int copy = newlazykid(oldId, l, r, 0);
        push(copy, l, r); int mid = (l + r) >> 1;
        int le = update(nodes[copy].le, l, mid, u
            , v, val),
            ri = update(nodes[copy].ri, mid + 1,
                r, u, v, val);
        return newparent(le, ri);
    }

    int update(int root, int u, int v, ll val) {
        assert(n > 0);
        return update(root, 1, n, u, v, val);
    }

    ll get(int id, int l, int r, int u, int v, ll
         carry = 0) {
        if (l > v || r < u) return 0;
        if (u <= l && r <= v) return nodes[id].
            sum + carry * (r - l + 1);
        carry += nodes[id].lazy; int mid = (l + r
            ) >> 1;
        return get(nodes[id].le, l, mid, u, v,
            carry) + get(nodes[id].ri, mid + 1, r
            , u, v, carry);
    }

    ll get(int root, int u, int v) {
        assert(n > 0);
        return get(root, 1, n, u, v);
    }
} mytree;
```

ConvexHullTrickMonotonicSlopes.h

Description: Fast convex hull trick for monotonic slopes
Time: $\mathcal{O}(1)$ for addition and $\mathcal{O}(1)$ or $\mathcal{O}(\log N)$ for query
Status: Well-tested

6042b6, 63 lines

```
// Adding decreasing slopes, query min
// Use get() if query increasing, query()
    otherwise
struct ConvexHullTrick {
    const ll INF = 1e18;
    struct Line {
        ll m, b;
```

```
    Line(ll _m = 0, ll _b = 0) {
        m = _m, b = _b;
    }

    ll eval(ll x) {
        return m * x + b;
    }
};

bool isBad(Line l1, Line l2, Line l3) {
    // bad when x of (l1 x l3) < x of (l1 x
        l2)
    return ((__int128)l3.b - l1.b) * ((
        __int128)l1.m - l2.m)
         <= ((__int128)l2.b - l1.b) * ((
            __int128)l1.m - l3.m);
}

vector<Line> lines;
int pointer = 0;

ConvexHullTrick() {
    lines.clear(), pointer = 0;
}

void reset() {
    lines.clear(), pointer = 0;
}

void add(ll m, ll b) { // insert line
    decreasing in slope
    while (!lines.empty() && lines.back().m
        == m && lines.back().b > b)
        lines.pop_back();
    if (!lines.empty() && lines.back().m == m
        ) return;
    Line newLine(m, b);
    while (lines.size() > 1 && isBad(lines[
        lines.size() - 2], lines.back(),
        newLine))
        lines.pop_back();
    lines.emplace_back(newLine);
}

ll query(ll x) { // return min
    if (lines.empty()) return INF;
    if (lines.size() == 1) return lines[0].
        eval(x);
    int low = 0, high = int(lines.size()) -
        2, best = -1;
    while (low <= high) {
        int mid = (low + high) >> 1;
        if (lines[mid + 1].eval(x) < lines[
            mid].eval(x)) low = (best = mid)
            + 1;
        else high = mid - 1;
    }
    return lines[best + 1].eval(x);
}

ll get(ll x) { //answer query increasing in x
    if (lines.empty()) return INF;
    if (pointer >= lines.size()) pointer =
        lines.size() - 1;
    while (pointer + 1 < lines.size() &&
        lines[pointer + 1].eval(x) < lines[
        pointer].eval(x))
        ++pointer;
    return lines[pointer].eval(x);
}
} cht;
```

FenwickTree2d.h

Description: Computes sums a[i,j] for all 0<=i<=I, 0<=j<=J, and increases single elements a[i,j] (0-index).
Usage: Requires update positions to be known in advance
in form vector<pair<int,int>> as constructor for FT2, or construct with limX only and call fakeUpdate().
Time: $\mathcal{O}\left(\log^2 N\right)$.
Status: stress-tested

47b6a8, 55 lines

```
struct FT {
  vector<ll> s;
  FT(int n) : s(n) {}
  void update(int pos, ll dif) { // a[pos] += dif
    for (; pos < sz(s); pos |= pos + 1) s[pos] +=
        dif;
  }
  ll query(int pos) { // sum of values in [0, pos
      )
    ll res = 0;
    for (; pos; pos -= (pos & (-pos))) res += s[
        pos - 1];
    return res;
  }
  int lower_bound(ll sum) {// min pos st sum of
      [0, pos] >= sum
    // Returns n if no sum is >= sum, or -1 if
        empty sum is.
    if (sum <= 0) return -1;
    int pos = 0;
    for (int pw = 1 << 25; pw; pw >>= 1) {
      if (pos + pw <= sz(s) && s[pos + pw - 1] <
          sum)
        pos += pw, sum -= s[pos - 1];
    }
    return pos;
  }
};

struct FT2 {
  vector<vi> ypos; vector<FT> ybit;
    bool call_init = false;
  void fakeUpdate(int x, int y) {
    for (; x < sz(ypos); x |= x + 1) ypos[x].
        push_back(y);
  }
  FT2(int limx, vector<pair<int,int>> &updates){
        ypos.resize(limx + 5);
    for(auto [x, y]: updates) fakeUpdate(x, y);
    init();
  }
  FT2(int limx) : ypos(limx + 5) {}
  void init() {
        call_init = true;
        ybit.clear();
```

```
    for (vi& v : ypos) sort(all(v)), ybit.
        emplace_back(sz(v));
  }
  int get_id(int x, int y) {
    return (int)(lower_bound(all(ypos[x]), y) -
        ypos[x].begin());
    }
  void update(int x, int y, ll dif) {
        if(!call_init) init();
    for (; x < sz(ypos); x |= x + 1) ybit[x].
        update(get_id(x, y), dif);
  }
  ll query(int x, int y) { //calculates sum for
     <=x, <=y.
        if(!call_init) init();
        x++;
    ll sum = 0;
    for (; x; x -= (x & (-x))) sum += ybit[x -
        1].query(upper_bound(all(ypos[x - 1]), y)
         - ypos[x - 1].begin());
    return sum;
  }
};
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

94c495, 8 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest
    ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return
      __builtin_bswap64(x*C); }
};
//to declare a hash table ht, do this.
__gnu_pbds::gp_hash_table<ll,int,chash> ht({},{},
    {},{},{1<<16});
```

ImplicitTreap.h

Description: An implicit treap on an array that supports deleting an element, inserting an element in any position and splitting the array into two parts (prefix/suffix).

Usage: Declare pnode t = nullptr for an empty array,
then use the functions. If you want to handle more data in each node,
then reimplement refresh(). Note that the array is 1-indexed.

Time: $\mathcal{O}(\log N)$

Status: Tested on one problem

e7be67, 59 lines

```
mt19937_64 rng(6736);
struct node{
    int x, sz = 1;
    uint64_t y;
    node *le = nullptr, *ri = nullptr;
    node(int x): x(x), y(rng()) {}
    node(int x, int y): x(x), y(y) {}
};
namespace implicit_treap{ //1-index
    using pnode = node*;
    int get_sz(pnode &p){
        if(p == nullptr) return 0;
        return p->sz;
    }
    void refresh(pnode &p){
        p->sz = get_sz(p->le) + get_sz(p->ri) +
            1;
    }
    //first is [1, i], second is [i + 1, n]
    pair<pnode, pnode> split(pnode t, int i){
        if(t == nullptr) return {nullptr, nullptr
            };
        pair<pnode, pnode> ans;
        if(get_sz(t->le) + 1 <= i){
            ans = split(t->ri, i - get_sz(t->le)
                - 1);
            t->ri = ans.first; ans.first = t;
            refresh(ans.first);
        }
        else{
            ans = split(t->le, i);
            t->le = ans.second; ans.second = t;
            refresh(ans.second);
        }
        return ans;
    }
    //t1 is treap of a1,...,an. t2 is treap of b1
        ,...,bm.
    //Merge into a1,...,an,b1,...,bm.
    pnode merge(pnode t1, pnode t2){
        if(t1 == nullptr) return t2;
        if(t2 == nullptr) return t1;
        if(t1->y >= t2->y){
            t1->ri = merge(t1->ri, t2);
            refresh(t1); return t1;
        }
        t2->le = merge(t1, t2->le);
        refresh(t2); return t2;
    }
    //add element x after i-th index
    void insert(pnode &t, int i, int x){
        pnode p = new node(x);
        auto [t1, t2] = split(t, i);
        t = merge(merge(t1, p), t2);
    }
    //remove i-th index
    void erase(pnode &t, int i){
        auto [t1, mid] = split(t, i - 1);
        auto [tx, t2] = split(mid, 1);
        t = merge(t1, t2);
        delete tx;
    }
}
```

LazySegmentTree.h

Description: Segment tree with ability to add or set values of large intervals, and compute max of intervals. Can be changed to other things. Use with a bump allocator for better performance, and SmallPtr or implicit indices to save memory.

Usage: Node* tr = new Node(v, 0, sz(v));

Time: $\mathcal{O}(\log N)$.

Status: stress-tested a bit

34ecf5, 50 lines

```
const int inf = 1e9;
struct Node {
```

```
Node *l = 0, *r = 0;
int lo, hi, mset = inf, madd = 0, val = -inf;
Node(int lo,int hi):lo(lo),hi(hi){} // Large
    interval of -inf
Node(vi& v, int lo, int hi) : lo(lo), hi(hi) {
  if (lo + 1 < hi) {
    int mid = lo + (hi - lo)/2;
    l = new Node(v, lo, mid); r = new Node(v,
        mid, hi);
    val = max(l->val, r->val);
  }
  else val = v[lo];
}
int query(int L, int R) {
  if (R <= lo || hi <= L) return -inf;
  if (L <= lo && hi <= R) return val;
  push();
  return max(l->query(L, R), r->query(L, R));
}
void set(int L, int R, int x) {
  if (R <= lo || hi <= L) return;
  if (L <= lo && hi <= R) mset = val = x, madd
      = 0;
  else {
    push(), l->set(L, R, x), r->set(L, R, x);
    val = max(l->val, r->val);
  }
}
void add(int L, int R, int x) {
  if (R <= lo || hi <= L) return;
  if (L <= lo && hi <= R) {
    if (mset != inf) mset += x;
    else madd += x;
    val += x;
  }
  else {
    push(), l->add(L, R, x), r->add(L, R, x);
    val = max(l->val, r->val);
  }
}
void push() {
  if (!l) {
    int mid = lo + (hi - lo)/2;
    l = new Node(lo, mid); r = new Node(mid, hi
        );
  }
  if (mset != inf)
    l->set(lo,hi,mset), r->set(lo,hi,mset),
        mset = inf;
  else if (madd)
    l->add(lo,hi,madd), r->add(lo,hi,madd),
        madd = 0;
 }
};
```

Matrix.h

Description: Matrix multiplication modulo MOD. Stored entries must be in [0, MOD).

Usage: `Set MOD; 8 * MOD * MOD must fit in signed 64-bit.`

Status: Randomized multiplication and exponentiation regression test

570e22, 79 lines

```
struct Matrix {
    static const ll MOD = 1000000007; // Set to
        the required modulus.
    static const uint64_t modmod = 8ULL * MOD *
        MOD; // must be < 2^63

    vector<vector<uint64_t>> val;
    int n, m;

    Matrix(int _n = 0, int _m = 0) {
        resize(_n, _m);
    }

    Matrix(vector<vector<ll>> ve) {
        init(ve);
    }

    void resize(int _n, int _m) {
        n = _n, m = _m;
        val.assign(n, vector<uint64_t>(m));
    }

    void init(vector<vector<ll>> ve) {
        n = ve.size(), m = n ? ve[0].size() : 0;
        val.assign(n, vector<uint64_t>(m, 0));
        for (int i = 0; i < n; ++i) for (int j =
            0; j < m; ++j) {
            val[i][j] = (ve[i][j] % MOD + MOD) %
                MOD;
        }
    }

    friend istream& operator >> (istream &is,
        Matrix &T) {
        for (auto &x : T.val) {
            for (auto &y : x) {
                ll v; if (!(is >> v)) return is;
                y = (v % MOD + MOD) % MOD;
            }
        }
        return is;
    }

    friend ostream& operator << (ostream &os,
        Matrix T) {
        for (auto &x : T.val) {
            for (auto &y : x) os << (int)(y % MOD
                ) << ' ';
            os << '\n';
        }
        return os << '\n';
    }

    vector<uint64_t>& operator [] (int i) {
        return val[i]; }
    const vector<uint64_t>& operator [] (int i)
        const { return val[i]; }

    Matrix operator * (const Matrix &other) {
        assert(m == other.n);
        Matrix ret(n, other.m);
        for (int i = 0; i < n; ++i) {
            for (int k = 0; k < m; ++k) {
                for (int j = 0; j < other.m; ++j)
                    {
                    ret[i][j] += val[i][k] *
                        other[k][j];
```

```
                }
                if (k % 8 == 0) {
                    for (int j = 0; j < other.m;
                        ++j) {
                        ret[i][j] = min(ret[i][j
                            ], ret[i][j] - modmod
                            );
                    }
                }
            }
        }
        for (auto &row : ret.val) for (auto &x :
            row) x %= MOD;
        return ret;
    }

    Matrix binpow(ll n) {
        assert(this->n == m && n >= 0);
        Matrix A = *this, ret(A.n, A.n);
        for (int i = 0; i < A.n; ++i) ret[i][i] =
             1;
        while (n > 0) {
            if (n & 1) ret = ret * A;
            A = A * A, n >>= 1;
        }
        return ret;
    }
};
```

MoQueries.h

Description: Using Mo's Algorithm to answer tree path (vertex) or interval queries, adding/removing one element each step.
Usage: change block size and Type of ans in t_ans and blsz.
Define add, remove and calc in namespace MoAlgo after including, then call mo_inter() or mo_tree().
Note that the array/tree is 0-index.
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$
Status: Tested on one tree problem and one interval problem.
ca8d29, 90 lines

```
namespace MoAlgo{
    using T_ans = ll;
    const int blsz = 350; // block size
    //(inter[0], inter[1]) is the current
        interval/path BEFORE OPERATION
    void add(int idx, array<int, 2> &inter); //
        add idx
    void remove(int idx, array<int, 2> &inter);
        // remove idx
    T_ans calc(array<int, 2> &inter);
    // compute current answer

    pii get_block(pii que){
        if((que.first / blsz) & 1) que.second = -
            que.second;
        return {que.first / blsz, que.second};
    }
    inline vector<T_ans> mo_inter(vector<pii> que
        ){
        array<int, 2> inter = {0, 0};
        add(0, inter);
        int &L = inter[0], &R = inter[1];
        vi order(sz(que));
        vector<T_ans> ans(sz(que));
        iota(all(order), 0);
        auto cmp = [&](int s, int t){
            return get_block(que[s]) < get_block(
                que[t]);
        };
        sort(all(order), cmp);
        for (int qi : order){
            pii q = que[qi];
            while(L > q.first){
                add(L - 1, inter); L--;
            }
            while(R < q.second){
                add(R + 1, inter); R++;
            }
            while(L < q.first){
                remove(L, inter); L++;
            }
            while(R > q.second){
                remove(R, inter); R--;
            }
            ans[qi] = calc(inter);
        }
        return ans;
    }
    inline vector<T_ans> mo_tree(vector<pii> que,
         vector<vi>& graph, int root=0){
        int n = sz(graph);
        array<int, 2> path = {root, root};
        vi order(sz(que)), in(n), out(n), on_path
            (n), pa(n);
        vector<T_ans> ans(sz(que));
        int tim = 0;
        add(root, path), on_path[root] = 1;
        auto dfs = [&](int u, int p, auto& self)
            -> void {
            pa[u] = p;
            in[u] = tim; tim++;
            for (int v : graph[u]) if (v != p)
                self(v, u, self);
            out[u] = tim; tim++;
        };
        dfs(root, -1, dfs);
        iota(all(order), 0);
        auto cmp = [&](int s, int t){
            pii ps = {in[que[s].first], in[que[s
                ].second]};
            pii pt = {in[que[t].first], in[que[t
                ].second]};
            return get_block(ps) < get_block(pt);
        };
        sort(all(order), cmp);
        for(int qi: order){
            vi tmp;
            tmp.reserve(n);
            auto move_end = [&](int &a, int b){
                auto step = [&](int c){
                    if (on_path[c]){
                        remove(a, path); on_path[
                            a] = 0;
                    }
                    else{
                        add(c, path); on_path[c]
                            = 1;
                    }
                    a = c;
```

```
            };
            for(;!(in[b] <= in[a] && in[a] <=
                 out[b]); b = pa[b]){
                tmp.push_back(b);
            }
            while (a != b) step(pa[a]);
            for(int i = (int)tmp.size() - 1;
                i >= 0; i--) step(tmp[i]);
            tmp.clear(),tmp.resize(0);
        };
        move_end(path[0], que[qi].first);
        move_end(path[1], que[qi].second);
        ans[qi] = calc(path);
    }
    return ans;
  }
}
```

RMQ.h

Description: Range Minimum Queries on an array. Returns min(V[a], V[a + 1], ... V[b - 1]) in constant time.
Usage: `RMQ rmq(values);`
`rmq.query(inclusive, exclusive);`
Time: $\mathcal{O}(|V| \log |V| + Q)$
Status: stress-tested

510c32, 16 lines

```
template<class T>
struct RMQ {
  vector<vector<T>> jmp;
  RMQ(const vector<T>& V) : jmp(1, V) {
    for (int pw = 1, k = 1; pw * 2 <= sz(V); pw
        *= 2, ++k) {
      jmp.emplace_back(sz(V) - pw * 2 + 1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j] = min(jmp[k - 1][j], jmp[k -
            1][j + pw]);
    }
  }
  T query(int a, int b) {
    assert(a < b); // or return inf if a == b
    int dep = 31 - __builtin_clz(b - a);
    return min(jmp[dep][a], jmp[dep][b - (1 <<
        dep)]);
  }
};
```

SegmentTreeLazyIterative.h

Description: Non-recursion segment tree with range updates and range queries.
Usage: `T, U are the node and lazy types. TT, UT, UU represents merging two nodes, applying lazy to a node, and merging two lazies.`
Time: $\mathcal{O}(\log N)$ for each operation
Status: Tested on range add update and range min query.

f55804, 85 lines

```
template<class T,class U> struct segtree{
    //fill merge node + node here
    T TT(T &le, T &ri);

    //fill apply lazy "app" variable to "node"
        variable here
    void UT(U &app, T &node);

    //Append new lazy "node" to existing lazy "
        app", assigning to "app"
    void UU(U &app, U &node);

    //Tid, Uid are default values of T, U,
    //representing empty node and empty lazy (no
        change).
    T Tid;
    U Uid;
    segtree(T _Tid, U _Uid, vector<T> &a){
        Tid = _Tid, Uid = _Uid;
        build(a);
    }
    vector<T> seg;
    vector<U> d;
    int n, lg;
    void refresh(int v){
        seg[v] = TT(seg[v<<1], seg[(v<<1)|1]);
    }
    //0-index
    void build(vector<T> &a){
        int siz = a.size();
        for(n = 1; n < siz;) n <<= 1;
        seg.assign(n<<1, Tid), d.assign(n, Uid),
            lg = __lg(n);
        for(int i = 0; i < siz; i++) seg[n + i] =
             a[i];
        for(int i = n - 1; i >= 1; i--) refresh(i
            );
    }
    void apply(int v, U &f){
        UT(f, seg[v]);
        if(v < n) UU(d[v], f);
    }
    void push(int v){
        apply(v << 1, d[v]), apply((v << 1) | 1,
            d[v]);
        d[v] = Uid;
    }
    void update(int l, int r, U &f){ //update in
        [l, r]
        l += n, r += (n + 1);
        if(l >= r) return;
        for(int i = lg; i >= 1; i--){
            if((l >> i << i) != l) push(l >> i);
            if((r >> i << i) != r) push((r - 1)
                >> i);
        }
        int l0 = l, r0 = r;
        for(;l < r; l >>= 1, r >>= 1){
            if(l & 1){
                apply(l, f);
                l++;
            }
            if(r & 1){
                r--;
                apply(r, f);
            }
        }
        l = l0, r = r0;
        for(int i = 1; i <= lg; i++){
            if((l >> i << i) != l) refresh(l >> i
                );
            if((r >> i << i) != r) refresh((r -
                1) >> i);
        }
```

```
    }
    T query(int l, int r){ // query in [l, r]
        l += n, r += (n + 1);
        if(l >= r) return Tid;
        for(int i = lg; i >= 1; i--){
            if((l >> i << i) != l) push(l >> i);
            if((r >> i << i) != r) push((r - 1)
                >> i);
        }
        T le = Tid, ri = Tid;
        for(;l < r; l >>= 1, r >>= 1){
            if(l & 1){
                le = TT(le, seg[l]);
                l++;
            }
            if(r & 1){
                r--;
                ri = TT(seg[r], ri);
            }
        }
        return TT(le, ri);
    }
};
```

SplayTree.h

Description: Implicit splay tree with insertion, deletion and access by 1-based sequence position.
Time: $\mathcal{O}(\log N)$ amortized per operation
Status: Tested on SPOJ

fd6655, 120 lines

```
struct Node {
    Node *parent, *child[2];
    int sz = 0, value;

    Node(int val = 0) {
        parent = nullptr, sz = 1, value = val;
        child[0] = child[1] = nullptr;
    }

    int side() {
        return parent ? parent->child[1] == this
            : -1;
    }

    void pushUp() {
        sz = 1;
        for (int i = 0; i < 2; ++i)
            if (child[i]) sz += child[i]->sz;
    }
};
const int NUM_NODES = 1e6 + 5;
Node nodes[NUM_NODES];
int numNode = 0;

Node *newNode(int value = 0) {
    assert(numNode < NUM_NODES);
    nodes[numNode] = Node(value);
    return &nodes[numNode++];
}

struct SplayTree {
    Node *root;

    SplayTree() {
        root = newNode(0);
        link(root, newNode(0), 1);
        root->pushUp();
    }

    Node *oneDir(int dir) {
        Node *p = root;
        while (p && p->child[dir]) p = p->child[
            dir];
        return p;
    }

    void link(Node *a, Node *b, int dir) {
        if (a == nullptr) {
            root = b;
            if (root) root->parent = nullptr;
            return;
        }
        a->child[dir] = b;
        if (b) b->parent = a;
    }

    void rotate(Node *a) {
        Node *p = a->parent;
        int dir = a->side();
        link(p->parent, a, p->side());
        link(p, a->child[!dir], dir);
        link(a, p, !dir);
        p->pushUp(), a->pushUp();
    }

    void splay(Node *a, Node *s = nullptr) {
        while (a->parent != s) {
            Node *p = a->parent;

            if (p->parent == s) rotate(a);
            else if (a->side() == p->side())
                rotate(p), rotate(a); // zig-zig
            else rotate(a), rotate(a);
        }
    }

    // Zero-based rank including sentinels;
        target must be below s if s != nullptr.
    void find(int k, Node *s = nullptr) {
        assert(0 <= k && k < root->sz);
        Node *p = root;
        while (true) {
            int cntLeft = (p->child[0] ? p->child
                [0]->sz : 0);
            if (cntLeft > k) p = p->child[0];
            else if (cntLeft + 1 > k) {
                splay(p, s); return;
            }
            else {
                k -= cntLeft + 1;
                p = p->child[1];
            }
        }
    }

    // Insert before the current k-th element. 1
       <= k <= n + 1
    void insert(int k, int val) {
        assert(1 <= k && k <= root->sz - 1);
        find(k);
```

```
        find(k - 1, root);

        Node *foo = newNode(val);
        link(foo, root->child[0], 0);
        link(root, foo, 0);
        foo->pushUp(), root->pushUp();
    }

    // 1 <= k <= n
    void remove(int k) {
        assert(1 <= k && k <= root->sz - 2);
        find(k + 1);
        find(k - 1, root);

        link(root->child[0], nullptr, 1);
        root->child[0]->pushUp();
        root->pushUp();
    }

    // 1 <= k <= n
    int get(int k) {
        assert(1 <= k && k <= root->sz - 2);
        find(k);
        return root->value;
    }
} tree;
```

TimeSegmentTree.h

Description: A time segment tree, used to support deletion in a DS that supports addition and rollback with an additional $\log N$ factor. Note that the queries from the DS has to be offline.

Usage: 1: Edit MAX_TIME, ope, Q (description below)
2: Define add(), rollback() and calc() in namespace segtree_time after including.
3: Turn the addition/deletion updates into tuples [x,l,r],
which means perform update x at time l and delete at time r + 1.
4: Call update(l, r, x) on all tuples, then call solve_queries()
to solve the queries offline.
rollback(k) to rollback k times, unite() and Findset()
to merge and find root.
Time: $\mathcal{O}(T(n) \log N)$, where $T(n)$ is the time complexity for the DS.
Status: Tested on one problem

39cb7b, 55 lines

```
namespace segtree_time{ //time is 1-indexed
    //ope is the update type put into the segtree
    //Q is the type of the query answers.
    using ope = pii;
    using Q = int;
    //MAX_TIME is the maximum time allowed
    const int MAX_TIME = 3e5;
    vector<vector<ope>> group(4 * MAX_TIME + 5);
    void update(int v, int TL, int TR, int l, int
        r, const ope &x){
        if(r < TL || l > TR) return;
        if(l <= TL && TR <= r){
            group[v].push_back(x); return;
        }
        int TM = ((TL + TR) >> 1);
        update(v << 1, TL, TM, l, r, x);
        update((v << 1) | 1, TM + 1, TR, l, r, x)
            ;
    }
    void update(int l, int r, const ope &x){
        assert(1 <= l && l <= r && r <= MAX_TIME)
            ;
        update(1, 1, MAX_TIME, l, r, x);
    }
    Q calc(int i); // Define to answer the i-th
        query.
    void add(ope &x); // Define to apply x to
        your DS.
    //rollback the last k operations of your DS
        here.
    //Note that ALL operations are counted, EVEN
        IF they don't change the DS.
    void rollback(int k);
    inline void DFS(int v, int TL, int TR, vector
        <pii> &que, int &ptr, vector<Q> &ans){
        if(ptr == que.size()) return;
        for(ope &x: group[v]) add(x);
        if(TL != TR){
            int TM=((TL + TR) >> 1);
            DFS(v << 1, TL, TM, que, ptr, ans);
            DFS((v << 1) | 1,TM + 1, TR, que, ptr
                , ans);
        }
        else{
            for(; ptr < que.size() && que[ptr].
                first == TL; ptr++){
                ans[que[ptr].second] = calc(que[
                    ptr].second);
            }
        }
        rollback(group[v].size());
    }
    //a query is of format {time_of_query,
        index_of_query}
    inline vector<Q> solve_queries(vector<pii> &
        queries){
        for (auto [time, index] : queries) {
            assert(1 <= time && time <= MAX_TIME)
                ;
            assert(0 <= index && index < (int)
                queries.size());
        }
        sort(queries.begin(), queries.end());
        int ptr = 0;
        vector<Q> ans(queries.size());
        DFS(1, 1, MAX_TIME, queries, ptr, ans);
        return ans;
```

```
    }
};
using namespace segtree_time;
```

Treap.h

Description: An explicit treap that supports split(), merge(), insert(), and erase().
Usage: Declare pnode t = nullptr for an empty treap,
then use the functions. If you want to handle more data in each node,
then reimplement get_val() and refresh().
Time: $\mathcal{O}(\log N)$
Status: Tested on two problems

635afb, 61 lines

```
mt19937_64 rng(6736);
struct node{
    int x, sz = 1;
    uint64_t y;
    node *le = nullptr, *ri = nullptr;
    node(int x): x(x), y(rng()) {}
    node(int x, int y): x(x), y(y) {}
};
namespace treap{
    using pnode = node*;
    int get_val(pnode &p){
        if(p == nullptr) return 0;
        return p->sz;
    }
    void refresh(pnode &p){
        p->sz = get_val(p->le) + get_val(p->ri) +
            1;
    }
    //first is <=x, second is >x
    pair<pnode, pnode> split(pnode t, int x, bool
        inclusive = true){
        if(t == nullptr) return {nullptr, nullptr
            };
        pair<pnode, pnode> ans;
        if(t->x < x || (inclusive && t->x == x)){
            ans = split(t->ri, x, inclusive);
            t->ri = ans.first; ans.first = t;
            refresh(ans.first);
        }
        else{
            ans = split(t->le, x, inclusive);
            t->le = ans.second; ans.second = t;
            refresh(ans.second);
        }
        return ans;
    }
    //assume all t1->x < t2->x (ordering between
        trees)
    pnode merge(pnode t1, pnode t2){
        if(t1 == nullptr) return t2;
        if(t2 == nullptr) return t1;
        if(t1->y >= t2->y){
            t1->ri = merge(t1->ri, t2);
            refresh(t1); return t1;
        }
        t2->le = merge(t1, t2->le);
        refresh(t2); return t2;
    }
    void insert(pnode &t, int x){
        pnode p = new node(x);
        auto [t1, t2] = split(t, x);
        t = merge(merge(t1, p), t2);
    }
    void destroy(pnode t){
        if (!t) return;
        destroy(t->le); destroy(t->ri); delete t;
    }
    //erase ALL occurences of x; O(log N + number
        removed)
    void erase(pnode &t, int x){
        auto [t1, mid] = split(t, x, false);
        auto [tx, t2] = split(mid, x);
        t = merge(t1, t2);
        destroy(tx);
    }
}
```

UnionFindRollback.h

Description: DSU with undo operations. DsuRollback undoes a count of unite calls; RollbackUF restores a saved time.

Usage: Call get_sz() to get size of cc, rollback(k) to undo k unite calls, including unsuccessful calls, unite() and Findset() to merge and find root.
Time: $\mathcal{O}(\log N)$
Status: Tested on two problems

91024f, 72 lines

```
struct node{
  int dsu = -1;
};
struct DsuRollback {
  vector<node> dsu; vector<pair<int, node>> st;
    int cc;
  DsuRollback(int n) : dsu(n + 5), cc(n) {}
  int get_sz(int i){
    return -dsu[Findset(i)].dsu;
  }
  int Findset(int i){
    if(dsu[i].dsu < 0) return i;
    return Findset(dsu[i].dsu);
  }
  void rollback(int k){
        assert(0 <= k && 2 * k <= (int)st.size())
            ;
        while (k--) {
            auto [v, oldv] = st.back(); st.
                pop_back();
            auto [u, oldu] = st.back(); st.
                pop_back();
            if (u == -1) continue;
            dsu[u] = oldu; dsu[v] = oldv; ++cc;
        }
  }
  //return false if already united, true
      otherwise.
  bool unite(int u, int v){
    u = Findset(u), v = Findset(v);
    if (u == v) {
            st.push_back({-1, {}}); st.push_back(
                {-1, {}});
            return false;
        }
        cc--;
    if (dsu[u].dsu < dsu[v].dsu) swap(u, v);
```

```
    st.push_back({u, dsu[u]});
    st.push_back({v, dsu[v]});
    dsu[v].dsu += dsu[u].dsu; dsu[u].dsu = v;
    return true;
  }
};

// Snapshot-based API used by DirectedMST (
    rollback takes a saved time).
/*
 * Author: Lukas Polacek, Simon Lindholm
 * Date: 2019-12-26
 * License: CC0
 * Source: folklore
 * Description: Disjoint-set data structure with
     undo.
 * If undo is not needed, skip st, time() and
     rollback().
 * Usage: int t = uf.time(); ...; uf.rollback(t);
 * Time: $O(\log(N))$
 * Status: tested as part of DirectedMST.h
 */
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[
      x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  }
};
```

Graph (7)

7.1 Graph theory

7.1.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--, mat[b][b]++` (and `mat[b][a]--, mat[a][a]++` if G is undirected). Remove the i-th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.1.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

7.2 Algorithms

TwoSat.h

Description: 2sat -> finding an arbitrary equation
Time: $\mathcal{O}(n+m)$
Status: Tested by KickingKun

485abb, 66 lines

```
struct TwoSat { // convert vector -> array to get
    higher speed ....
  int n; vector <int> lab, num, low; vector <
      vector <int>> adj;
  int timeDfs, scc; stack <int> st;

  TwoSat (int _n) {
    this->n = _n; lab.assign(n * 2 + 1, 0); low.
        assign(n * 2 + 1, 0);
    num.assign(n * 2 + 1, 0); adj.assign(n * 2 +
        1, vector <int>());
    timeDfs = scc = 0; while (!st.empty()) st.pop
        ();
  }

  int NON(int u) {
    return u <= n ? u + n : u - n;
  }

  void add(int u, int v) {
    adj[u].emplace_back(v);
  }

  void add_u_or_v(int u, int v) { // u or v = 1
    adj[NON(u)].emplace_back(v); adj[NON(v)].
        emplace_back(u);
  }
  void assign(int u, bool val) { // u = val
    if (val == 1) add_u_or_v(u, u);
    else add_u_or_v(NON(u), NON(u));
  }
  void add_equal(int u, int v) { // u = v
    add_u_or_v(u, NON(v)); add_u_or_v(NON(u), v);
  }
  void add_non_equal(int u, int v) { // u != v
    add_u_or_v(u, v); add_u_or_v(NON(u), NON(v));
  }

  void dfs(int u) {
    low[u] = num[u] = ++timeDfs; st.emplace(u);
    for (int v: adj[u]) {
      if (lab[v]) continue;
      if (!num[v]) dfs(v), low[u] = min(low[u],
          low[v]);
      else low[u] = min(low[u], num[v]);
    }

    if (low[u] == num[u]) {
      ++scc;
      while (true) {
        int v = st.top(); st.pop();
        lab[v] = scc; if (v == u) break;
      }
    }
```

```
  }

  bool check_exist() {
    for (int i = 1; i <= n; i++) {
      if (!lab[NON(i)]) dfs(NON(i));
      if (!lab[i]) dfs(i);
    }
    for (int i = 1; i <= n; i++)
      if (lab[i] == lab[NON(i)]) return false;
    return true;
  }

  vector <int> get_equation() {
    if (!check_exist()) return {-1};
    vector <int> ans = {0};
    for (int i = 1; i <= n; i++) ans.emplace_back
        (lab[i] < lab[NON(i)]);
    return ans;
  }
};
```

BipartieMatching.h

Description: use for max matching
Time: $\mathcal{O}\left(E \cdot \sqrt{V}\right)$
Status: approved by KickingKun

b7e4ae, 82 lines

```
struct bipartite_matching {
  const int n, m;
  vector <int> match1, match2, que, dist;
  vector <vector<int>> adj;

  bipartite_matching(int _n, int _m) : n(_n), m(
      _m) {
    match1.assign(n + 1, -1); match2.assign(m +
        1, -1);
    que.assign(n + 1, 0); dist.assign(n + 1, 0);
        adj.assign(n + 1, vector <int>());
  }

  void add(int u, int v) {
    adj[u].emplace_back(v);
  }

  bool bfs() {
    fill(dist.begin(), dist.end(), -1);
    int queBegin = 1, queEnd = 1;
    for (int u = 1; u <= n; ++u) {
      if (match1[u] == -1) dist[u] = 0, que[
          queEnd++] = u;
    }
    bool success = false;
    while (queBegin < queEnd) {
      int u = que[queBegin++];
      for (int v : adj[u]) {
        if (match2[v] == -1) {
          success = true;
        }
        else if (dist[match2[v]] == -1) {
          dist[match2[v]] = dist[u] + 1;
          que[queEnd++] = match2[v];
        }
      }
    }
    return success;
  }

  bool dfs(int u) {
    for (int v : adj[u]) {
      if (match2[v] == -1 || (dist[match2[v]] ==
          dist[u] + 1 && dfs(match2[v]))) {
        match1[u] = v; match2[v] = u; dist[u] = n
            + m;
        return true;
      }
    }
    dist[u] = n + m;
    return false;
  }

  int max_matching() {
    while (bfs()) {
      for (int u = 1; u <= n; ++u) {
        if (match1[u] == -1) dfs(u);
      }
    }
    return n - count(match1.begin() + 1, match1.
        end(), -1);
  }

  pair <vector <int>, vector <int>>
      minimum_vertex_cover() {
      vector <int> L, R;
      for (int u = 1; u <= n; ++u) {
        if (dist[u] == -1) L.emplace_back(u);
        else if (match1[u] != -1) R.emplace_back(
            match1[u]);
      }
    return {L, R};
  }

    pair <vector <int>, vector <int>>
        maximum_independent_set() {
    auto [_L, _R] = minimum_vertex_cover();
    vector <int> L, R;
    vector <bool> mark1(n + 2), mark2(m + 2);
    for (int x: _L) mark1[x] = true; for (int x:
        _R) mark2[x] = true;
    for (int u = 1; u <= n; u++) if (!mark1[u]) L
        .emplace_back(u);
    for (int u = 1; u <= m; u++) if (!mark2[u]) R
        .emplace_back(u);
    return {L, R};
  }

  vector <pii> get_edges() { // get from max
      matching
      vector <pii> ans;
      for (int u = 1; u <= n; ++u)
          if (match1[u] != -1) ans.emplace_back(u
              , match1[u]);
      return ans;
  }
};
```

MaxFlow.h

Description: Dinitz's algorithm to find max-flow.
Usage: `Call myflow.resize(n) to initalize the network.`

Time: $\mathcal{O}\left(V^2 \cdot E\right)$ in worst cases
Status: Well-tested

3be25f, 72 lines

```
struct Dinic {
    struct Edge {
        int u, v;
        ll capacity, flow = 0;

        Edge(int _u = 0, int _v = 0, ll _capacity
             = 0) {
            u = _u, v = _v, capacity = _capacity,
                 flow = 0;
        }
    };

    const ll FLOW_INF = (ll)2e18;
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> level, ptr;
    int n, m;

    Dinic(int n = 0) { resize(n); }

    void resize(int _n) {
        n = _n, m = 0;
        adj.assign(n + 1, vector<int>()), edges.
            clear();
    }

    int add(int u, int v, ll capacity) {
        edges.emplace_back(u, v, capacity);
        edges.emplace_back(v, u, 0);
        adj[u].emplace_back(m++), adj[v].
            emplace_back(m++);
        return m - 2;
    }

    ll dfs(int u, int sink, ll pushed) {
        if (u == sink || pushed == 0) return
            pushed;
        for (int &i = ptr[u]; i < (int)adj[u].
            size(); ++i) {
            int id = adj[u][i]; Edge e = edges[id
                ];
            if (level[u] + 1 != level[e.v] || e.
                capacity - e.flow <= 0) continue;
            if (ll p = dfs(e.v, sink, min(pushed,
                 e.capacity - e.flow))) {
                edges[id].flow += p, edges[id ^
                    1].flow -= p;
                return p;
            }
        }
        return 0;
    }

    ll maxFlow(int source, int sink) {
        assert(source != sink);
        ll flow = 0;
        do {
            level = ptr = vector<int>(n + 1, 0);
            queue<int> q; q.emplace(source);
                level[source] = 1;
            while (!q.empty()) {
                int u = q.front(); q.pop();
                for (auto id : adj[u]) {
                    if (level[edges[id].v] ||
                        edges[id].capacity -
                        edges[id].flow <= 0)
                        continue;
                    level[edges[id].v] = level[u]
                         + 1;
                    q.emplace(edges[id].v);
                }
            }
            while (ll pushed = dfs(source, sink,
                FLOW_INF)) flow += pushed;
        } while (level[sink] != 0);
        return flow;
    }

    vector<pii> minCut() { // remember to call
        maxFlow(source, sink) and be careful of
        duplicates
        vector<pii> cut;
        for (int id = 0; id < m; id += 2) {
            auto &e = edges[id];
            if (e.capacity > 0 && level[e.u] && !
                level[e.v])
                cut.emplace_back(e.u, e.v);
        }
        return cut;
    }
} myflow;
```

MinCostMaxFlow.h

Description: Edmond-Karp algorithm with SPFA to find the shortest path.
Usage: `Call myflow.resize(n) to initalize the network.`
Time: $\mathcal{O}\left(F \cdot V \cdot E\right)$ in worst cases
Status: Well-tested

357dcc, 74 lines

```
struct MCMF {
    struct Edge {
        int u, v;
        ll capacity, cost, flow = 0;

        Edge(int _u = 0, int _v = 0, ll _capacity
             = 0, ll _cost = 0) {
            u = _u, v = _v, capacity = _capacity,
                 cost = _cost, flow = 0;
        }
    };

    const ll FLOW_INF = (ll)2e18;
    vector<Edge> edges;
    vector<vector<int>> adj;
    vector<int> level, ptr;
    int n, m;

    MCMF(int n = 0) { resize(n); }

    void resize(int _n) {
        n = _n, m = 0;
        adj.assign(n + 1, vector<int>()), edges.
            clear();
    }

    int add(int u, int v, ll capacity, ll cost) {
        edges.emplace_back(u, v, capacity, cost);
```

```
    edges.emplace_back(v, u, 0, -cost);
    adj[u].emplace_back(m++), adj[v].
        emplace_back(m++);
    return m - 2;
}

bool spfa(int source, int sink, vector<ll> &
    dist, vector<int> &par) {
    dist.assign(n + 1, FLOW_INF), par.assign(
        n + 1, -1);
    vector<bool> inQueue(n + 1, false); queue
        <int> q;
    dist[source] = 0; inQueue[source] = true;
         q.emplace(source);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        inQueue[u] = false;
        for (auto id : adj[u]) {
            int v = edges[id].v;
            if (edges[id].capacity - edges[id
                ].flow <= 0) continue;
            if (dist[v] > dist[u] + edges[id
                ].cost) {
                dist[v] = dist[u] + edges[id
                    ].cost;
                par[v] = id;
                if (!inQueue[v]) {
                    inQueue[v] = true;
                    q.emplace(v);
                }
            }
        }
    }
    return dist[sink] != FLOW_INF;
}

pair<ll, ll> minCost(int source, int sink) {
    assert(source != sink);
    ll flow = 0, cost = 0; vector<ll> dist;
        vector<int> par;
    while (spfa(source, sink, dist, par)) {
        ll amount = FLOW_INF - flow; int cur
            = sink;
        while (cur != source) {
            amount = min(amount, edges[par[
                cur]].capacity - edges[par[
                cur]].flow);
            cur = edges[par[cur]].u;
        }

        flow += amount, cost += amount * dist
            [sink], cur = sink;
        while (cur != source) {
            edges[par[cur]].flow += amount;
            edges[par[cur] ^ 1].flow -=
                amount;
            cur = edges[par[cur]].u;
        }
    }

    return make_pair(flow, cost);
  }
} myflow;
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`

Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses.

Status: stress-tested

5f9706, 56 lines

```
struct TwoSat {
  int N;
  vector<vi> gr;
  vi values; // 0 = false, 1 = true

  TwoSat(int n = 0) : N(n), gr(2*n) {}

  int addVar() { // (optional)
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f, int j) {
    f = max(2*f, -1-2*f);
    j = max(2*j, -1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x) { either(x, x); }

  void atMostOne(const vi& li) { // (optional)
    if (sz(li) <= 1) return;
    int cur = ~li[0];
    rep(i,2,sz(li)) {
      int next = addVar();
      either(cur, ~li[i]);
      either(cur, next);
      either(~li[i], next);
      cur = ~next;
    }
    either(cur, ~li[1]);
  }

  vi val, comp, z; int time = 0;
  int dfs(int i) {
    int low = val[i] = ++time, x; z.push_back(i);
    for(int e : gr[i]) if (!comp[e])
      low = min(low, val[e] ?: dfs(e));
    if (low == val[i]) do {
      x = z.back(); z.pop_back();
      comp[x] = low;
      if (values[x>>1] == -1)
        values[x>>1] = x&1;
    } while (x != i);
    return val[i] = low;
  }
```

```
  bool solve() {
    values.assign(N, -1);
    val.assign(2*N, 0); comp = val;
    rep(i,0,2*N) if (!comp[i]) dfs(i);
    rep(i,0,N) if (comp[2*i] == comp[2*i+1])
        return 0;
    return 1;
  }
};
```

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$.
Time: $\mathcal{O}(VE)$
Status: Tested on kattis:shortestpath3

830a8f, 23 lines

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a :
    -a; }};
struct Node { ll dist = inf; int prev = -1; };

void bellmanFord(vector<Node>& nodes, vector<Ed>&
    eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() <
      b.s(); });

  int lim = sz(nodes) / 2 + 2; // /3+100 with
      shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    }
  }
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  }
}
```

BiconnectedComponents.h

Description: Finds all biconnected components in an undirected graph, and runs a callback for the edges in each. In a biconnected component there are at least two internally disjoint paths between any two nodes (a cycle exists through them). Note that a node can be in several components. An edge which is not in a component is a bridge, i.e., not part of any cycle.
Usage: `int eid = 0; ed.resize(N);`
`for each edge (a,b) {`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...});`
Time: $\mathcal{O}(E+V)$
Status: tested during MIPT ICPC Workshop 2017

c6b7c7, 32 lines

```
vi num, st;
vector<vector<pii>> ed;
int Time;
template<class F>
int dfs(int at, int par, F& f) {
  int me = num[at] = ++Time, top = me;
  for (auto [y, e] : ed[at]) if (e != par) {
    if (num[y]) {
      top = min(top, num[y]);
      if (num[y] < me)
        st.push_back(e);
    } else {
      int si = sz(st);
      int up = dfs(y, e, f);
      top = min(top, up);
      if (up == me) {
        st.push_back(e);
        f(vi(st.begin() + si, st.end()));
        st.resize(si);
      }
      else if (up < me) st.push_back(e);
      else { /* e is a bridge */ }
    }
  }
  return top;
}
template<class F>
void bicomps(F f) {
  num.assign(sz(ed), 0);
  rep(i,0,sz(ed)) if (!num[i]) dfs(i, -1, f);
}
```

BinaryLifting.h

Description: Calculate power of two jumps in a tree, to support fast upward jumps and LCAs. Assumes the root node points to itself.
Time: construction $\mathcal{O}(N \log N)$, queries $\mathcal{O}(\log N)$
Status: Tested at Petrozavodsk, also stress-tested via LCA.cpp

bfce85, 25 lines

```
vector<vi> treeJump(vi& P){
  int on = 1, d = 1;
  while(on < sz(P)) on *= 2, d++;
  vector<vi> jmp(d, P);
  rep(i,1,d) rep(j,0,sz(P))
    jmp[i][j] = jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>& tbl, int nod, int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i)) nod = tbl[i][nod];
  return nod;
}

int lca(vector<vi>& tbl, vi& depth, int a, int b)
    {
  if (depth[a] < depth[b]) swap(a, b);
  a = jmp(tbl, a, depth[a] - depth[b]);
  if (a == b) return a;
  for (int i = sz(tbl); i--;) {
    int c = tbl[i][a], d = tbl[i][b];
    if (c != d) a = c, b = d;
  }
  return tbl[0][a];
}
```

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S|\log|S|)$
Status: Tested at CodeForces

"LCA.h" bb56cc, 22 lines

```
typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  if (subset.empty()) return {};
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[
      b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  }
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  }
  return ret;
}
```

DFSMatching.h

Description: Simple bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.
Usage: `vi btoa(m, -1); dfsMatching(g, btoa);`
Time: $\mathcal{O}(VE)$
Status: works

522b98, 22 lines

```
bool find(int j, vector<vi>& g, vi& btoa, vi& vis
    ) {
  if (btoa[j] == -1) return 1;
  vis[j] = 1; int di = btoa[j];
  for (int e : g[di])
    if (!vis[e] && find(e, g, btoa, vis)) {
      btoa[e] = di;
      return 1;
    }
  return 0;
}
int dfsMatching(vector<vi>& g, vi& btoa) {
  vi vis;
  rep(i,0,sz(g)) {
    vis.assign(sz(btoa), 0);
    for (int j : g[i])
      if (find(j, g, btoa, vis)) {
        btoa[j] = i;
        break;
      }
  }
  return sz(btoa) - (int)count(all(btoa), -1);
}
```

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.
Status: Tested on SPOJ FASTFLOW and SPOJ MATCHING, stress-tested

d7f0f1, 42 lines

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if
        you need flows
  };
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap,
        rcap});
  }
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster
        for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (Edge e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] +
                1;
      }
      while (ll p = dfs(s, t, LLONG_MAX)) flow +=
          p;
    } while (lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a) { return lvl[a] != 0;
      }
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E\log V)$

Status: Stress-tested, also tested on NWERC 2018 fastest-speedrun

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  Edge top() { prop(); return key; }
};
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
}
void pop(Node*& a) { a->prop(); a = merge(a->l, a
    ->r); }

pair<ll, vi> dmst(int n, int r, vector<Edge>& g)
    {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b],
      new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]])
            ;
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] =
            -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end
            ]}});
      }
    }
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  }

  for (auto& [u,t,comp] : cycs) { // restore sol
      (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  }
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
}
```

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D, computes a $(D+1)$-coloring of the edges such that no neighboring edges share a color. (D-coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)
Time: $\mathcal{O}(NM)$
Status: stress-tested, tested on kattis:gamescheduling

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds) {
  vi cc(N + 1), ret(sz(eds)), fan(N), free(N),
      loc;
  for (pii e : eds) ++cc[e.first], ++cc[e.second
      ];
  int u, v, ncols = *max_element(all(cc)) + 1;
  vector<vi> adj(N, vi(ncols, -1));
  for (pii e : eds) {
    tie(u, v) = e;
    fan[0] = v;
    loc.assign(ncols, 0);
    int at = u, end = u, d, c = free[u], ind = 0,
        i = 0;
    while (d = free[v], !loc[d] && (v = adj[u][d
        ]) != -1)
      loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
    cc[loc[d]] = c;
    for (int cd = d; at != -1; cd ^= c ^ d, at =
        adj[at][cd])
      swap(adj[at][cd], adj[end = at][cd ^ c ^ d
          ]);
    while (adj[fan[i]][d] != -1) {
      int left = fan[i], right = fan[++i], e = cc
          [i];
      adj[u][e] = left;
      adj[left][e] = u;
      adj[right][e] = -1;
      free[right] = e;
    }
    adj[u][d] = fan[i];
    adj[fan[i]][d] = u;
    for (int y : {fan[0], u, end})
      for (int& z = free[y] = 0; adj[y][z] != -1;
          z++);
  }
  rep(i,0,sz(eds))
    for (tie(u, v) = eds[i]; adj[u][ret[i]] != v
        ;) ++ret[i];
  return ret;
}
```

EdmondsKarp.h

Description: Flow algorithm with guaranteed complexity $O(VE^2)$. To get edge flow values, compare capacities before and after, and take the positive values only.
Status: stress-tested

482fe0, 36 lines

```
template<class T> T edmondsKarp(vector<
    unordered_map<int, T>>&
    graph, int source, int sink) {
  assert(source != sink);
  T flow = 0;
```

```
  vi par(sz(graph)), q = par;

  for (;;) {
    fill(all(par), -1);
    par[source] = 0;
    int ptr = 1;
    q[0] = source;

    rep(i,0,ptr) {
      int x = q[i];
      for (auto e : graph[x]) {
        if (par[e.first] == -1 && e.second > 0) {
          par[e.first] = x;
          q[ptr++] = e.first;
          if (e.first == sink) goto out;
        }
      }
    }
    return flow;
out:
    T inc = numeric_limits<T>::max();
    for (int y = sink; y != source; y = par[y])
      inc = min(inc, graph[par[y]][y]);

    flow += inc;
    for (int y = sink; y != source; y = par[y]) {
      int p = par[y];
      if ((graph[p][y] -= inc) <= 0) graph[p].
          erase(y);
      graph[y][p] += inc;
    }
  }
}
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V+E)$
Status: stress-tested

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges,
    int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just
      cycles
  while (!s.empty()) {
    int x = s.back(), y, e, &it = its[x], end =
        sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back
        (); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }}
  for (int x : D) if (x < 0 || sz(ret) != nedges
      +1) return {};
  return {ret.rbegin(), ret.rend()};
}
```

FloydWarshall.h

Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m, where $m[i][j] = \texttt{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j, `inf` if no path, or `-inf` if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$
Status: slightly tested

531245, 12 lines

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m) {
  int n = sz(m);
  rep(i,0,n) m[i][i] = min(m[i][i], 0LL);
  rep(k,0,n) rep(i,0,n) rep(j,0,n)
    if (m[i][k] != inf && m[k][j] != inf) {
      auto newDist = max(m[i][k] + m[k][j], -inf)
          ;
      m[i][j] = min(m[i][j], newDist);
    }
  rep(k,0,n) if (m[k][k] < 0) rep(i,0,n) rep(j,0,
      n)
    if (m[i][k] != inf && m[k][j] != inf) m[i][j]
        = -inf;
}
```

GeneralMatching.h

Description: Matching for general graphs. Fails with probability N/mod.
Time: $\mathcal{O}(N^3)$
Status: not very well tested

"../numerical/MatrixInverse-mod.h" cb1912, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>&
    ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() %
        mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) %
            mod;
      }
    }
  } while (matInv(A = mat) != M);

  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
```

```
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j]
            * b) % mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

GlobalMinCut.h

Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $\mathcal{O}(V^3)$
Status: Stress-tested together with GomoryHu

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat) {
  pair<int, vi> best = {INT_MAX, {}};
  int n = sz(mat);
  vector<vi> co(n);
  rep(i,0,n) co[i] = {i};
  rep(ph,1,n) {
    vi w = mat[0];
    size_t s = 0, t = 0;
    rep(it,0,n-ph) { // O(V^2) -> O(E log V) with
        prio. queue
      w[t] = INT_MIN;
      s = t, t = max_element(all(w)) - w.begin();
      rep(i,0,n) w[i] += mat[t][i];
    }
    best = min(best, {w[t] - mat[t][t], co[t]});
    co[s].insert(co[s].end(), all(co[t]));
    rep(i,0,n) mat[s][i] += mat[t][i];
    rep(i,0,n) mat[i][s] = mat[s][i];
    mat[0][t] = INT_MIN;
  }
  return best;
}
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $\mathcal{O}(V)$ Flow Computations
Status: Tested on CERC 2015 J, stress-tested

"PushRelabel.h" 0418b3, 13 lines

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree;
  vi par(N);
  rep(i,1,N) {
    PushRelabel D(N); // Dinic also works
    for (Edge t : ed) D.addEdge(t[0], t[1], t[2],
        t[2]);
    tree.push_back({i, par[i], D.calc(i, par[i])}
        );
    rep(j,i+1,N)
      if (par[j] == par[i] && D.leftOfMinCut(j))
          par[j] = i;
  }
  return tree;
}
```

HLD.h

Description: Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. Code does additive modifications and max queries, but can support commutative segtree modifications/queries on paths and subtrees. Takes as input the full adjacency list. VALS_EDGES being true means that values are stored in the edges, as opposed to the nodes. All values initialized to the segtree default. Root must be 0.
Time: $\mathcal{O}\left((\log N)^2\right)$
Status: stress-tested against old HLD

"../data-structures/LazySegmentTree.h" 9547af, 46 lines

```
template <bool VALS_EDGES> struct HLD {
  int N, tim = 0;
  vector<vi> adj;
  vi par, siz, rt, pos;
  Node *tree;
  HLD(vector<vi> adj_)
    : N(sz(adj_)), adj(adj_), par(N, -1), siz(N,
      1),
    rt(N),pos(N),tree(new Node(0, N)){ dfsSz(0)
        ; dfsHld(0); }
void dfsSz(int v) {
  for (int& u : adj[v]) {
    adj[u].erase(find(all(adj[u]), v));
    par[u] = v;
    dfsSz(u);
    siz[v] += siz[u];
    if (siz[u] > siz[adj[v][0]]) swap(u, adj[v
        ][0]);
  }
}
void dfsHld(int v) {
  pos[v] = tim++;
  for (int u : adj[v]) {
    rt[u] = (u == adj[v][0] ? rt[v] : u);
    dfsHld(u);
  }
}
template <class B> void process(int u, int v, B
    op) {
  for (;; v = par[rt[v]]) {
    if (pos[u] > pos[v]) swap(u, v);
    if (rt[u] == rt[v]) break;
    op(pos[rt[v]], pos[v] + 1);
  }
  op(pos[u] + VALS_EDGES, pos[v] + 1);
}
void modifyPath(int u, int v, int val) {
  process(u, v, [&](int l, int r) { tree->add(l
      , r, val); });
}
int queryPath(int u, int v) { // Modify
    depending on problem
  int res = -1e9;
  process(u, v, [&](int l, int r) {
      res = max(res, tree->query(l, r));
  });
  return res;
}
```

```
  int querySubtree(int v) { // modifySubtree is
      similar
    return tree->query(pos[v] + VALS_EDGES, pos[v
        ] + siz[v]);
  }
};
```

hopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and *btoa* should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $btoa[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Usage: `vi btoa(m, -1); hopcroftKarp(g, btoa);`

Time: $\mathcal{O}\left(\sqrt{V}E\right)$

Status: stress-tested by MinimumVertexCover, and tested on oldkattis.adkbipmatch and SPOJ:MATCHING

f612e4, 42 lines

```
bool dfs(int a, int L, vector<vi>& g, vi& btoa,
    vi& A, vi& B) {
  if (A[a] != L) return 0;
  A[a] = -1;
  for (int b : g[a]) if (B[b] == L + 1) {
    B[b] = 0;
    if (btoa[b] == -1 || dfs(btoa[b], L + 1, g,
        btoa, A, B))
      return btoa[b] = a, 1;
  }
  return 0;
}

int hopcroftKarp(vector<vi>& g, vi& btoa) {
  int res = 0;
  vi A(g.size()), B(btoa.size()), cur, next;
  for (;;) {
    fill(all(A), 0);
    fill(all(B), 0);
    cur.clear();
    for (int a : btoa) if(a != -1) A[a] = -1;
    rep(a,0,sz(g)) if(A[a] == 0) cur.push_back(a)
        ;
    for (int lay = 1;; lay++) {
      bool islast = 0;
      next.clear();
      for (int a : cur) for (int b : g[a]) {
        if (btoa[b] == -1) {
          B[b] = lay;
          islast = 1;
        }
        else if (btoa[b] != a && !B[b]) {
          B[b] = lay;
          next.push_back(btoa[b]);
        }
      }
      if (islast) break;
      if (next.empty()) return res;
      for (int a : next) A[a] = lay;
      cur.swap(next);
    }
    rep(a,0,sz(g))
      res += dfs(a, 0, g, btoa, A, B);
  }
}
```

LCA.h

Description: Data structure for computing lowest common ancestors in a tree (with 0 as root). C should be an adjacency list of the tree, either directed or undirected.

Time: $\mathcal{O}(N \log N + Q)$

Status: stress-tested

"../data-structures/RMQ.h" 0f62fb, 21 lines

```
struct LCA {
  int T = 0;
  vi time, path, ret;
  RMQ<int> rmq;

  LCA(vector<vi>& C) : time(sz(C)), rmq((dfs(C
      ,0,-1), ret)) {}
  void dfs(vector<vi>& C, int v, int par) {
    time[v] = T++;
    for (int y : C[v]) if (y != par) {
      path.push_back(v), ret.push_back(time[v]);
      dfs(C, y, v);
    }
  }
  int lca(int a, int b) {
    if (a == b) return a;
    tie(a, b) = minmax(time[a], time[b]);
    return path[rmq.query(a, b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*
      depth[lca(a,b)];}
};
```

LinkCutTree.h

Description: Represents a forest of unrooted trees. You can add and remove edges (as long as the result is still a forest), and check whether two nodes are in the same tree.

Time: All operations take amortized $\mathcal{O}(\log N)$.

Status: Stress-tested a bit for N <= 20

0fb462, 90 lines

```
struct Node { // Splay tree. Root's pp contains
    tree's parent.
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  Node() { c[0] = c[1] = 0; fix(); }
  void fix() {
    if (c[0]) c[0]->p = this;
    if (c[1]) c[1]->p = this;
    // (+ update sum of subtree elements etc. if
        wanted)
  }
  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }
  int up() { return p ? p->c[1] == this : -1; }
  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z
        = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
```

```
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  Node* first() {
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this)
        ;
  }
};

struct LinkCut {
  vector<Node> node;
  LinkCut(int N) : node(N) {}

  void link(int u, int v) { // add an edge (u, v)
    assert(!connected(u, v));
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }
  void cut(int u, int v) { // remove an edge (u,
      v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else {
      x->c[0] = top->p = 0;
      x->fix();
    }
  }
  bool connected(int u, int v) { // are u, v in
      the same tree?
    Node* nu = access(&node[u])->first();
    return nu == access(&node[v])->first();
  }
  void makeRoot(Node* u) {
    access(u);
    u->splay();
    if(u->c[0]) {
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }
  Node* access(Node* u) {
    u->splay();
    while (Node* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) {
        pp->c[1]->p = 0; pp->c[1]->pp = pp; }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
};
```

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

Status: stress-tested

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X
    ={}, B R={}) {
  if (!P.any()) { if (!X.any()) f(R); return; }
  auto q = (P | X)._Find_first();
  auto cands = P & ~eds[q];
  rep(i,0,sz(eds)) if (cands[i]) {
    R[i] = 1;
    cliques(eds, f, P & eds[i], X & eds[i], R);
    R[i] = P[i] = 0; X[i] = 1;
  }
}
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.

Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs.

Status: stress-tested

f7c0bc, 49 lines

```
typedef vector<bitset<200>> vb;
struct Maxclique {
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v
        .i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d
        > b.d; });
    int mxD = r[0].d;
    rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.
          push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) -
            sz(q) + 1, 1);
```

```
      C[1].clear(), C[2].clear();
      for (auto v : T) {
        int k = 1;
        auto f = [&](int i) { return e[v.i][i];
            };
        while (any_of(all(C[k]), f)) k++;
        if (k > mxk) mxk = k, C[mxk + 1].clear
            ();
        if (k < mnk) T[j++].i = v.i;
        C[k].push_back(v.i);
      }
      if (j > 0) T[j - 1].d = 0;
      rep(k,mnk,mxk + 1) for (int i : C[k])
        T[j].i = i, T[j++].d = k;
      expand(T, lev + 1);
    } else if (sz(q) > sz(qmax)) qmax = q;
    q.pop_back(), R.pop_back();
  }
 }
 vi maxClique() { init(V), expand(V); return
     qmax; }
 Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(
     C)), old(S) {
   rep(i,0,sz(e)) V.push_back({i});
 }
};
```

MaximumIndependentSet.h

Description: To obtain a maximum independent set of a graph, find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.

MinCut.h

Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

Status: works

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

Status: stress-tested

"DFSMatching.h" da4196, 20 lines

```
vi cover(vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = dfsMatching(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] =
      false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e]
        != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
}
```

PushRelabel.h

Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.

Time: $\mathcal{O}\left(V^2\sqrt{E}\right)$

Status: Tested on Kattis and SPOJ, and stress-tested

0ae1d4, 48 lines

```
struct PushRelabel {
  struct Edge {
    int dest, back;
    ll f, c;
  };
  vector<vector<Edge>> g;
  vector<ll> ec;
  vector<Edge*> cur;
  vector<vi> hs; vi H;
  PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*
      n), H(n) {}

  void addEdge(int s, int t, ll cap, ll rcap=0) {
    if (s == t) return;
    g[s].push_back({t, sz(g[t]), 0, cap});
    g[t].push_back({s, sz(g[s])-1, 0, rcap});
  }

  void addFlow(Edge& e, ll f) {
    Edge &back = g[e.dest][e.back];
    if (!ec[e.dest] && f) hs[H[e.dest]].push_back
        (e.dest);
    e.f += f; e.c -= f; ec[e.dest] += f;
    back.f -= f; back.c += f; ec[back.dest] -= f;
  }
  ll calc(int s, int t) {
    int v = sz(g); H[s] = v; ec[t] = 1;
    vi co(2*v); co[0] = v-1;
    rep(i,0,v) cur[i] = g[i].data();
    for (Edge& e : g[s]) addFlow(e, e.c);

    for (int hi = 0;;) {
      while (hs[hi].empty()) if (!hi--) return -
          ec[s];
      int u = hs[hi].back(); hs[hi].pop_back();
      while (ec[u] > 0)  // discharge u
        if (cur[u] == g[u].data() + sz(g[u])) {
          H[u] = 1e9;
          for (Edge& e : g[u]) if (e.c && H[u] >
              H[e.dest]+1)
            H[u] = H[e.dest]+1, cur[u] = &e;
          if (++co[H[u]], !--co[hi] && hi < v)
            rep(i,0,v) if (hi < H[i] && H[i] < v)
              --co[H[i]], H[i] = v + 1;
          hi = H[u];
        } else if (cur[u]->c && H[u] == H[cur[u
            ]->dest]+1)
          addFlow(*cur[u], min(ec[u], cur[u]->c))
              ;
        else ++cur[u];
```

```
    }
  }
  bool leftOfMinCut(int a) { return H[a] >= sz(g)
      ; }
};
```

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E+V)$
Status: Bruteforce-tested for N <= 5

76b5c9, 24 lines

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F
    & f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j);
    f(cont); cont.clear();
    ncomps++;
  }
  return val[j] = low;
}
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
}
```

TopoSort.h

Description: Topological sorting. Given is an oriented graph. Output is an ordering of vertices, such that there are edges only from left to right. If there are cycles, the returned list will have size smaller than n – nodes reachable from cycles will not be returned.
Time: $\mathcal{O}(|V|+|E|)$
Status: stress-tested

d678d8, 8 lines

```
vi topoSort(const vector<vi>& gr) {
  vi indeg(sz(gr)), q;
  for (auto& li : gr) for (int x : li) indeg[x
      ]++;
  rep(i,0,sz(gr)) if (indeg[i] == 0) q.push_back(
      i);
  rep(j,0,sz(q)) for (int x : gr[q[j]])
    if (--indeg[x] == 0) q.push_back(x);
  return q;
}
```

WeightedMatching.h

Description: Given a weighted bipartite graph, matches every node on the left with a node on the right such that no nodes are in two matchings and the sum of the edge weights is minimal. Takes cost[N][M], where cost[i][j] = cost for L[i] to be matched with R[j] and returns (min cost, match), where L[i] is matched with R[match[i]]. Negate costs for max cost. Requires $N \leq M$.
Time: $\mathcal{O}\left(N^2M\right)$
Status: Tested on kattis:cordonbleu, stress-tested

1e0fe9, 31 lines

```
pair<int, vi> hungarian(const vector<vi> &a) {
  if (a.empty()) return {0, {}};
  int n = sz(a) + 1, m = sz(a[0]) + 1;
  vi u(n), v(m), p(m), ans(n - 1);
  rep(i,1,n) {
    p[0] = i;
    int j0 = 0; // add "dummy" worker 0
    vi dist(m, INT_MAX), pre(m, -1);
    vector<bool> done(m + 1);
    do { // dijkstra
      done[j0] = true;
      int i0 = p[j0], j1, delta = INT_MAX;
      rep(j,1,m) if (!done[j]) {
        auto cur = a[i0 - 1][j - 1] - u[i0] - v[j
            ];
        if (cur < dist[j]) dist[j] = cur, pre[j]
            = j0;
        if (dist[j] < delta) delta = dist[j], j1
            = j;
      }
      rep(j,0,m) {
        if (done[j]) u[p[j]] += delta, v[j] -=
            delta;
        else dist[j] -= delta;
      }
      j0 = j1;
    } while (p[j0]);
    while (j0) { // update alternating path
      int j1 = pre[j0];
      p[j0] = p[j1], j0 = j1;
    }
  }
  rep(j,1,m) if (p[j]) ans[p[j] - 1] = j - 1;
  return {-v[0], ans}; // min cost
}
```

Geometry (8)

Geometry.h

Description: Geometric Template
Time: $\mathcal{O}(1)$
Status: Seems OK

2dde4c, 136 lines

```
struct Point {
  double x, y;
  Point() {}
  Point (double _x, double _y) {
    this->x = _x; this->y = _y;
  }

  double len() {
    return x * x + y * y;
  }

  Point operator - (Point other) {
```

```
    return {x - other.x, y - other.y};
  }

  Point operator - () {
    return Point(-x, -y);
  }

  bool operator == (Point other) {
    return x == other.x && y == other.y;
  }

  bool operator != (Point other) {
    return x != other.x || y != other.y;
  }

  void show() {
    cout << x << ' ' << y << '\n';
  }
};

struct Line {
  double a, b, c;
  Line() {}
  Line (double _a, double _b, double _c) {
    this->a = _a; this->b = _b; this->c = _c;
  }

  void simplify() { // if double = ll
    double d = __gcd(__gcd((ll)a, (ll)b), (ll)c);
    a /= d, b /= d, c /= d;
    // else do nothing
  }
  void show() {
    cout << a << ' ' << b << ' ' << c << '\n';
  }
};

double dot(Point A, Point B) { // dot product
  return A.x * B.x + A.y * B.y;
}

double cross(Point A, Point B) { // cross product
  return A.x * B.y - A.y * B.x;
}

int sign(double x) {
  if (x < 0) return -1;
  if (x == 0) return 0;
  return 1;
}

double dis(Point A, Point B) {
  return sqrt((B.x - A.x) * (B.x - A.x) + (B.y -
      A.y) * (B.y - A.y));
}

double dis(Point m, Line d) { // d(M, delta)
  return abs(d.a * m.x + d.b * m.y + d.c) / sqrt(
      d.a * d.a + d.b * d.b);
}

double polygon_area(vector <Point> p) {
  if (p.size() < 3) return 0;
  double res = 0;
  for (int i = 1; i < p.size() - 1; i++)
    res += cross(p[i] - p[0], p[i + 1] - p[0]);
  return abs(res) / 2.0;
} // tested

bool same_side(Point A, Point B, Line d) { // out
    of line
  double y1 = d.a * A.x + d.b * A.y + d.c;
  double y2 = d.a * B.x + d.b * B.y + d.c;
  return (y1 > 0 && y2 > 0) || (y1 < 0 && y2 < 0)
      ;
} // tested by eyes

Line lineVector(Point A, Point n) { // Point +
    vector n
  return Line(n.x, n.y, -n.x * A.x - n.y * A.y);
}

Line line(Point A, Point B) { // Point A, B
  Point n(A.y - B.y, B.x - A.x);
  return Line(n.x, n.y, -n.x * A.x - n.y * A.y);
}

bool inside_line(Point A, Line d) { // inside of
    line
  return d.a * A.x + d.b * A.y + d.c == 0;
}

bool inside_segment(Point M, Point A, Point B) {
    // inside of segment AB
  if (M.x > max(A.x, B.x) || M.x < min(A.x, B.x))
      return false;
  if (M.y > max(A.y, B.y) || M.y < min(A.y, B.y))
      return false;
  return inside_line(M, line(A, B));
} // tested CSES / VNOI

Point intersect(Line A, Line B) { // find p(x, y)
  Point res; if (A.b == 0) swap(A, B);
  res.x = (B.c * A.b - A.c * B.b) / (A.a * B.b -
      B.a * A.b);
  res.y = (-A.c - A.a * res.x) / A.b;
  return res;
} // seem OK

Line perpendicular(Point A, Line d) {
  Point n(-d.b, d.a);
  return Line(n.x, n.y, -n.x * A.x - n.y * A.y);
} // tested

bool obtuse(Point M, Point A, Point B) { // MAB >
    90
  return (M - A).len() + (B - A).len() < (M - B).
      len();
} // tested

double dist(Point M, Point A, Point B) { // dis(M
    , segment AB)
if (A == B) return dis(M, A);
//  check MBA <= 90 && MAB <= 90
  if (obtuse(M, A, B) || obtuse(M, B, A)) return
      min(dis(M, A), dis(M, B));
  return dis(M, line(A, B));
} // tested at VNOI
```

```
bool intersect(Point A, Point B, Point C, Point D
    ) { // segment AB and CD
  int x = sign(cross(C - A, B - A)), y = sign(
      cross(D - A, B - A));
  int u = sign(cross(A - C, D - C)), v = sign(
      cross(B - C, D - C));
  if (x == 0 && inside_segment(C, A, B)) return
      true;
  if (y == 0 && inside_segment(D, A, B)) return
      true;
  if (u == 0 && inside_segment(A, C, D)) return
      true;
  if (v == 0 && inside_segment(B, C, D)) return
      true;
  return (x != y && u != v);
} // tested at CSES
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.
Time: $\mathcal{O}\left(n^2\right)$
Status: tested on SPOJ CH3D

"Point3D.h" 5b45fc, 49 lines

```
typedef Point3D<double> P3;

struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
};

struct F { P3 q; int a, b, c; };

vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {
      -1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  };
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      }
    }
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a,
     f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    }
  }
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c,
        it.b);
  return FS;
};
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.
Usage: vector<Angle> v = {w[0], w[0].t360() ...}; // sorted
int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }
// sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i
Status: Used, works well

0f0602, 35 lines

```
struct Angle {
  int x, y;
  int t;
  Angle(int x, int y, int t=0) : x(x), y(y), t(t)
      {}
  Angle operator-(Angle b) const { return {x-b.x,
      y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  }
  Angle t90() const { return {-y, x, t + (half()
      && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()
      }; }
  Angle t360() const { return {x, y, t + 1}; }
};
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare
      distances
  return make_tuple(a.t, a.half(), a.y * (ll)b.x)
      <
         make_tuple(b.t, b.half(), a.x * (ll)b.y)
             ;
}

// Given two points, this calculates the smallest
     angle between
// them, i.e., the angle that covers the defined
    line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b
    ) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
```

```
        make_pair(a, b) : make_pair(b, a.t360()
            ));
}
Angle operator+(Angle a, Angle b) { // point a +
    vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
}
Angle angleDiff(Angle a, Angle b) { // angle b -
    angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x,
      tu - (b < a)};
}
```

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.
Status: stress-tested

"Point.h" 84d6d3, 11 lines

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair
    <P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-
      r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1
             *r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax
      (0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
}
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.
Status: unit tested

"Point.h" e0cfba, 9 lines

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.
      dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.
      dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.
Time: $\mathcal{O}(n)$
Status: Tested on GNYR 2019 Gerrymandering, stress-tested

"../../content/geometry/Point.h" 19add1, 19 lines

```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r
        *r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -
        a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q
        ) * r2;
  };
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] -
        c);
  return sum;
}
```

CircleTangents.h

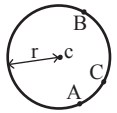
Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.
Status: tested

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2
    , double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 -
      dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) /
        d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  if (h2 == 0) out.pop_back();
  return out;
}
```

circumcircle.h

Description:
The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



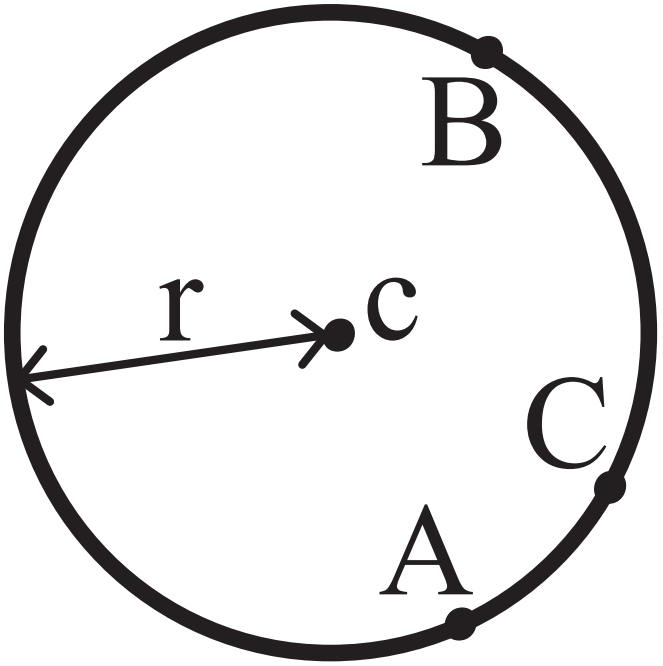
Status: tested

"Point.h" 1caa3a, 9 lines

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P&
    C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
```

```
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.
      cross(c)/2;
}
```

8.0.1 circumcircle



ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$
Status: stress-tested

"Point.h" ac41a6, 17 lines

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; }
      );
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}}
      ;
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.
        upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}
          });
    S.insert(p);
  }
  return ret.second;
}
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

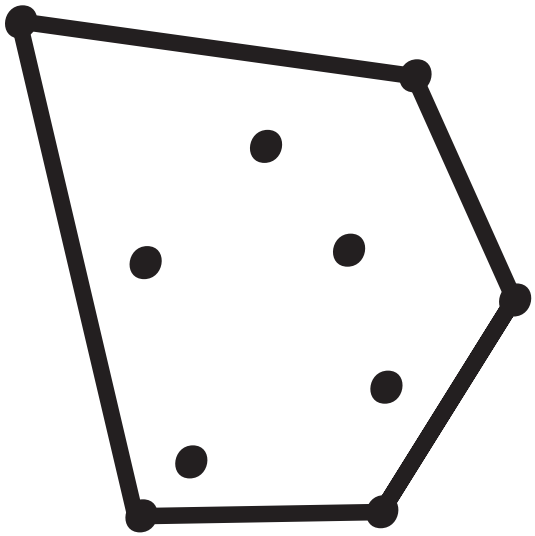


Time: $\mathcal{O}(n \log n)$
Status: stress-tested, tested with kattis:convexhull

"Point.h" 310954, 13 lines

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1);
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts
      )))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p
          ) <= 0) t--;
      h[t++] = p;
    }
  return {h.begin(), h.begin() + t - (t == 2 && h
      [0] == h[1])};
}
```

8.0.2 ConvexHull



DelaunayTriangulation.h

Description: Computes the Delaunay triangulation of a set of points. Each circumcircle contains none of the input points. If any three points are collinear or any four are on the same circle, behavior is undefined.
Time: $\mathcal{O}(n^2)$
Status: stress-tested

"Point.h", "3dHull.h" c0e7bc, 10 lines

```
template<class P, class F>
void delaunay(vector<P>& ps, F trifun) {
  if (sz(ps) == 3) { int d = (ps[0].cross(ps[1],
      ps[2]) < 0);
    trifun(0,1+d,2-d); }
  vector<P3> p3;
  for (P p : ps) p3.emplace_back(p.x, p.y, p.
      dist2());
```

```
  if (sz(ps) > 3) for(auto t:hull3d(p3)) if ((p3[
      t.b]-p3[t.a]).
       cross(p3[t.c]-p3[t.a]).dot(P3(0,0,1)) < 0)
    trifun(t.a, t.c, t.b);
}
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.

Time: $\mathcal{O}(n \log n)$

Status: stress-tested

"Point.h" eefdf5, 88 lines

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords
    are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any
    other point

struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c) { // is p in the
    circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.
      cross(c,a)*B > 0;
}
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new
      Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1
       ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
}
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b
      ->o);
}
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
}

pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s
        [1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c :
        b->r() };
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())
      ) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)
             ));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;
#define DEL(e, init, dir) Q e = init->dir; if (
    valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) {
        \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev())
        ;
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H
        (LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  }
  return { ra, rb };
}

vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts
      .end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e
      ->o;
#define ADD { Q c = e; do { c->mark = 1; pts.
    push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c
      != e); }
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark)
      ADD;
  return pts;
}
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$
Status: stress-tested, tested on kattis:roberthood

"Point.h" c571b8, 12 lines

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i
          ], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1]
          - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$
Status: stress-tested and tested on kattis:pointinpolygon

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11 lines

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict =
    true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return
        !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p
        [i], q) > 0;
  }
  return cnt;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)
Status: Tested on excellentengineers

"Point.h" bac5b0, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();

bool on_x(const P& a, const P& b) { return a.x <
    b.x; }
bool on_y(const P& a, const P& b) { return a.y <
    b.y; }

struct Node {
  P pt; // if this is a leaf, the single point in
      it
  T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; //
      bounds
  Node *first = 0, *second = 0;

  T distance(const P& p) { // min squared
      distance to a point
    T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
    T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
    return (P(x,y) - p).dist2();
  }

  Node(vector<P>&& vp) : pt(vp[0]) {
    for (P p : vp) {
      x0 = min(x0, p.x); x1 = max(x1, p.x);
      y0 = min(y0, p.y); y1 = max(y1, p.y);
    }
    if (vp.size() > 1) {
      // split on x if width >= height (not ideal
          ...)
      sort(all(vp), x1 - x0 >= y1 - y0 ? on_x :
          on_y);
      // divide by taking half the array for each
          child (not
      // best performance with many duplicates in
          the middle)
      int half = sz(vp)/2;
      first = new Node({vp.begin(), vp.begin() +
          half});
      second = new Node({vp.begin() + half, vp.
          end()});
    }
  }
};

struct KDTree {
  Node* root;
  KDTree(const vector<P>& vp) : root(new Node({
      all(vp)})) {}

  pair<T, P> search(Node *node, const P& p) {
    if (!node->first) {
      // uncomment if we should not find the
          point itself:
      // if (p == node->pt) return {INF, P()};
      return make_pair((p - node->pt).dist2(),
          node->pt);
    }

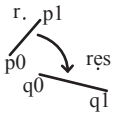
    Node *f = node->first, *s = node->second;
    T bfirst = f->distance(p), bsec = s->distance
        (p);
    if (bfirst > bsec) swap(bsec, bfirst), swap(f
        , s);

    // search closest side first, other side if
        needed
    auto best = search(f, p);
    if (bsec < best.first)
      best = min(best, search(s, p));
    return best;
  }

  // find nearest point to a point, and its
      squared distance
  // (requires an arbitrary operator< for Point)
  pair<T, P> nearest(const P& p) {
    return search(root, p);
  }
```

```
};
```

linearTransformation.h

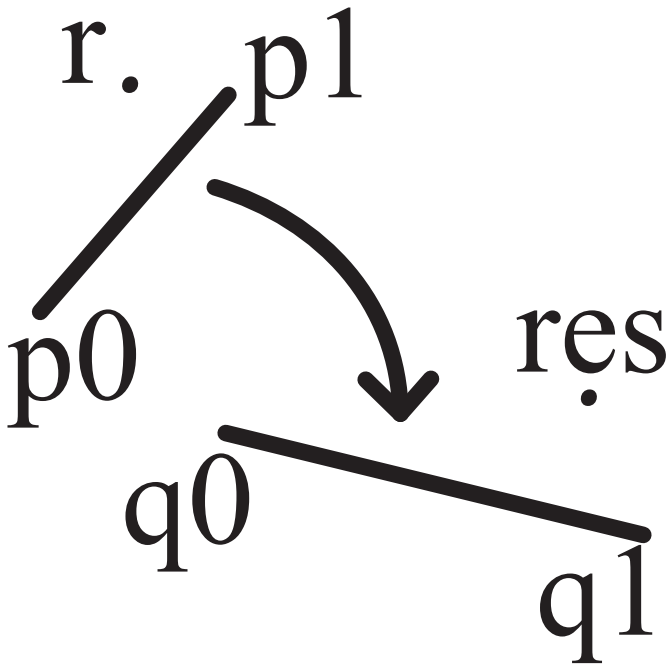


Description:
Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.
Status: not tested

"Point.h" 03a306, 6 lines

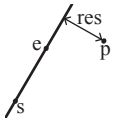
```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.
      dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num
      ))/dp.dist2();
}
```

8.0.3 linearTransformation



lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

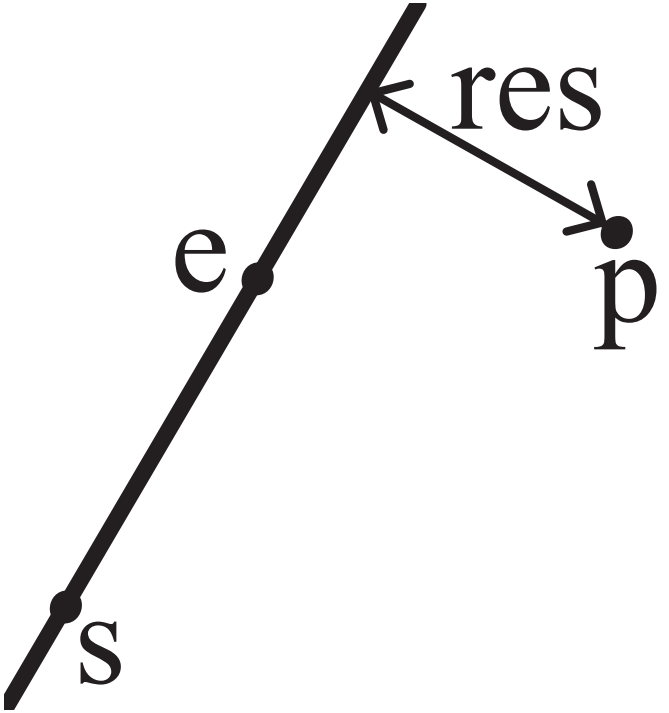


Status: tested

"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P& a, const P& b, const P&
    p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

8.0.4 lineDistance



LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: • $(-1,-1)$ if no collision, • $(i,-1)$ if touching the corner i, • (i,i) if along side $(i,i+1)$, • (i,j) if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$
Status: stress-tested

"Point.h" 7cf45b, 39 lines

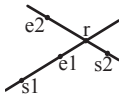
```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n
    ]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i -
    1 + n) < 0
```

```
template <class P> int extrVertex(vector<P>& poly
    , P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ?
        hi : lo) = m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly)
    {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2)
          % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    }
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  }
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz
        (poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    }
  return res;
}
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`

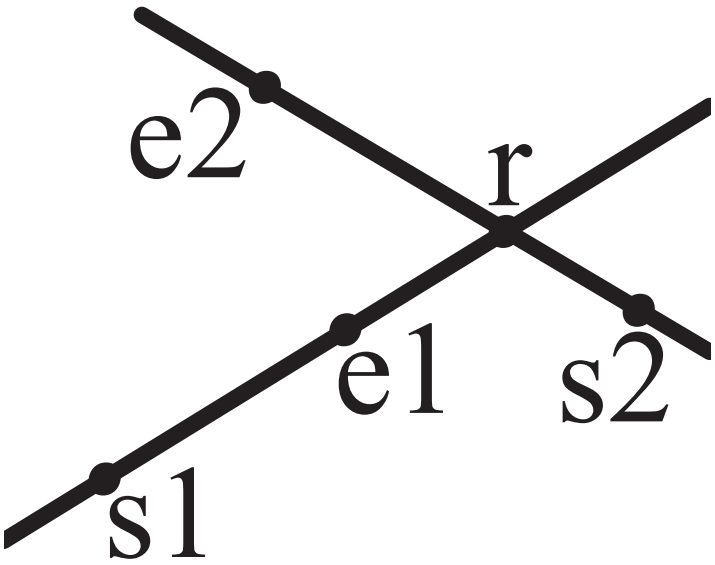
```
if (res.first == 1)
cout << "intersection point at " << res.second
<< endl;
```

Status: stress-tested, and tested through half-plane tests

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1)
      ;
  return {1, (s1 * p + e1 * q) / d};
}
```

8.0.5 lineIntersection



LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

Status: stress-tested

"Point.h" b5562d, 5 lines

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.
      dist2();
}
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$
Status: Stress-tested

"Point.h" df6f59, 23 lines

```
typedef Point<int> P;
vector<array<int, 3>> manhattanMST(vector<P> ps)
    {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<int, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y
             ;});
    map<int, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it
                    ++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      }
      sweep[-ps[i].y] = i;
    }
    for (P& p : ps) if (k & 1) p.x = -p.x; else
        swap(p.x, p.y);
  }
  return edges;
}
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$
Status: stress-tested

"circumcircle.h" 09dd0a, 17 lines

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r *
      EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS)
        {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS
          ) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      }
    }
  }
  return {o, r};
}
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use (segDist(s,e,p)<=epsilon) instead when using Point<double>.

"Point.h" c597e8, 3 lines

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p)
      <= 0;
}
```

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)
Status: Works fine, used a lot

47ec0a, 28 lines

```
template <class T> int sgn(T x) { return (x > 0)
    - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T x=0, T y=0) : x(x), y(y) {}
  bool operator<(P p) const { return tie(x,y) <
      tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==
      tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y)
      ; }
  P operator-(P p) const { return P(x-p.x, y-p.y)
      ; }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).
      cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2
      ()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } //
      makes dist()=1
  P perp() const { return P(-y, x); } // rotates
      +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around
       the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a))
        ; }
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")";
        }
};
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.
Status: tested, except for phi and theta

8058ae, 32 lines

```
template<class T> struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
```

```
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y
      (y), z(z) {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); }
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); }
  P operator+(R p) const { return P(x+p.x, y+p.y,
      z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y,
      z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d
      ); }
  P operator/(T d) const { return P(x/d, y/d, z/d
      ); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z
      ; }
  P cross(R p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y
        - y*p.x);
  }
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2
      ()); }
  //Azimuthal angle (longitude) to x-axis in
      interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  //Zenith angle (latitude) to the z-axis in
      interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*
      y),z); }
  P unit() const { return *this/(T)dist(); } //
      makes dist()=1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw
      around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u =
        axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*
        s;
  }
};
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$
Status: stress-tested

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict
    = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.
      back(), p);
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0],
      l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  }
  return sgn(l[a].cross(l[b], p)) < r;
}
```

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!
Status: Stress-tested and tested on kattis:polygonarea

"Point.h" f12300, 6 lines

```
template<class T>
T polygonArea2(vector<Point<T>>& v) {
  T a = v.back().cross(v[0]);
  rep(i,0,sz(v)-1) a += v[i].cross(v[i+1]);
  return a;
}
```

PolygonCenter.h

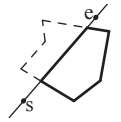
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$
Status: Tested

"Point.h" 9706dc, 9 lines

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i
      ++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  }
  return res / A / 3;
}
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

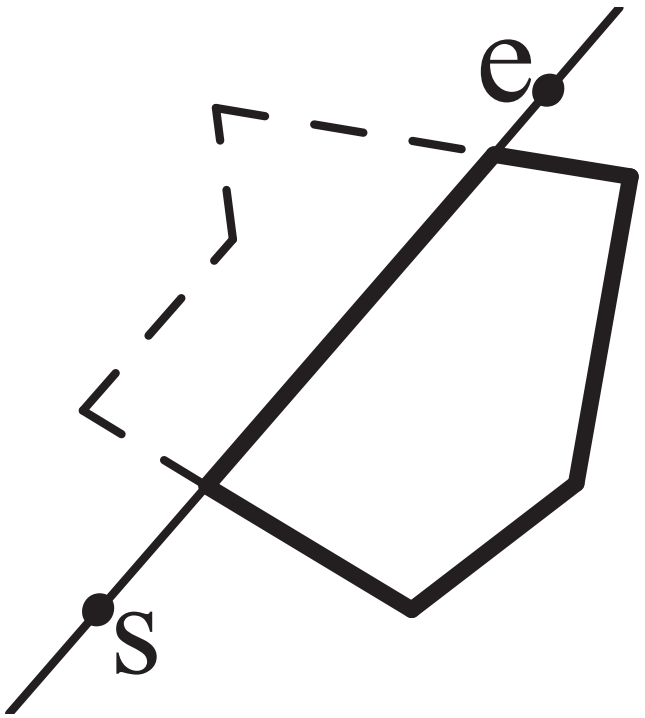


Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`
Status: tested but not extensively

"Point.h" d07181, 13 lines

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s,
    P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.
        back();
    auto a = s.cross(e, cur), b = s.cross(e, prev
        );
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a
          - b)));
    if (a < 0)
      res.push_back(cur);
  }
  return res;
}
```

8.0.6 PolygonCut



PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points
Status: stress-tested, Submitted on ECNA 2017 Problem A

"Point.h", "sideOf.h" 3931c6, 33 lines

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x
    : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(
        poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1,
         0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) %
            sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A,
            B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross
              (D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn
                (sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A)
            .dot(D-C))>0){
          segs.emplace_back(rat(C - A, B - A), 1)
              ;
          segs.emplace_back(rat(D - A, B - A),
              -1);
        }
      }
    }
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.
        first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j -
          1].first;
      cnt += segs[j].second;
    }
    ret += A.cross(B) * sum;
  }
  return ret / 2;
}
```

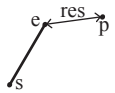
PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.
Status: tested

3058c3, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L&
    trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b
      ]).dot(p[i.c]);
  return v / 6;
}
```

SegmentDistance.h

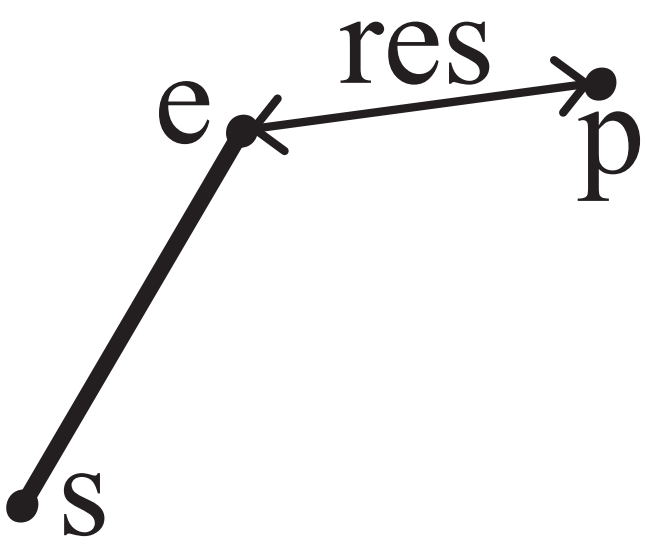


Description:
Returns the shortest distance between point p and the line segment from point s to e.
Usage: `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10;`
Status: tested

"Point.h" 5c88f4, 6 lines

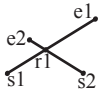
```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).
      dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

8.0.7 SegmentDistance



SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



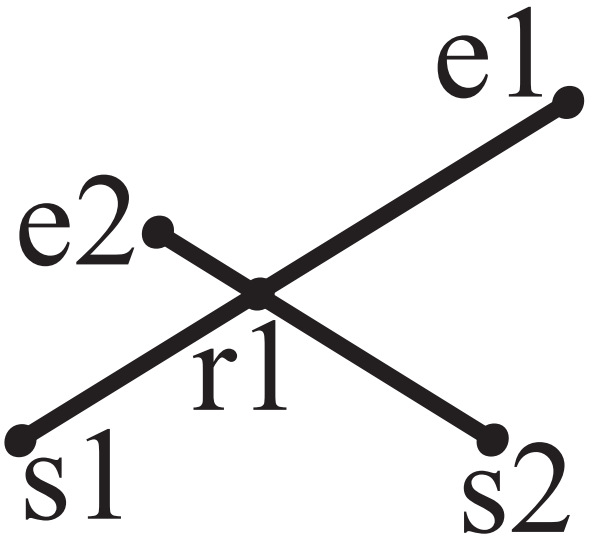
Usage: `vector<P> inter = segInter(s1,e1,s2,e2); if (sz(inter)==1) cout << "segments intersect at " << inter[0] << endl;`

Status: stress-tested, tested on kattis:intersection

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P
    c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-
      endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od)
      < 0)
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
}
```

8.0.8 SegmentIntersection



sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

Status: tested

"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e,
    p)); }

template<class P>
```

```
int sideOf(const P& s, const P& e, const P& p,
    double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
}
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.
Status: tested on kattis:airlinehub

611f07, 8 lines

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(d/2);
}
```

Strings (9)

AhoCorasick.h

Description: Aho-corasick.
Usage: `Call init() before any operations.`
Time: $\mathcal{O}(len(s))$ for addition, $\mathcal{O}(total_len)$ for build(), $\mathcal{O}(len(x))$ or $\mathcal{O}(numofoccurences)$ for query()
Status: Tested on some problems

9db8f2, 68 lines

```
struct AhoCorasick {
    static const int SIZE = 26;
    static const char START_CHAR = 'a';

    struct Node {
        int fail, link;
        int child[SIZE], nxt[SIZE];
        int output;

        Node() {
            fail = 0, link = -1, output = 0;
            memset(child, -1, sizeof child);
            memset(nxt, -1, sizeof nxt);
        }
    };

    vector<Node> nodes;

    void init() {
        nodes.clear(), nodes.emplace_back(Node())
            ;
    }

    void add(string &s, int id) {
        int v = 0;
        for (char ch : s) {
            int c = ch - START_CHAR;
            if (nodes[v].child[c] == -1) {
                nodes[v].child[c] = nodes.size();
                nodes.emplace_back(Node());
            }
            v = nodes[v].child[c];
        }
        ++nodes[v].output;
    }

    void build() {
        nodes[0].fail = nodes[0].link = 0;
        for (int i = 0; i < SIZE; ++i)
            nodes[0].nxt[i] = (nodes[0].child[i]
                == -1 ? 0 : nodes[0].child[i]);

        queue<int> q; q.emplace(0);
        while (!q.empty()) {
            int u = q.front(); q.pop();

            for (int c = 0; c < SIZE; ++c) if (
                nodes[u].child[c] != -1) {
                int v = nodes[u].child[c];
                nodes[v].fail = (u == 0 ? 0 :
                    nodes[nodes[u].fail].nxt[c]);
                for (int d = 0; d < SIZE; ++d)
                    nodes[v].nxt[d] = (nodes[v].
                        child[d] == -1 ? nodes[
                        nodes[v].fail].nxt[d] :
                        nodes[v].child[d]);
                if (nodes[nodes[v].fail].output
                    != 0) nodes[v].link = nodes[v
                    ].fail;
                else nodes[v].link = nodes[nodes[
                    v].fail].link;
                q.emplace(v);
            }
        }
    }

    int query(string &x) {
        int ans = 0, p = 0;
        for (auto ch : x) {
            int c = ch - START_CHAR, nxt = nodes[
                p].nxt[c];
            for (int u = nxt; u != 0; u = nodes[u
                ].link) {
                ans += nodes[u].output;
            }
            p = nxt;
        }
        return ans;
    }
} aho;
```

SuffixArray.h

Description: Suffix array using prefix doubling to sort with Kasai's algorithm for LCP
Time: $\mathcal{O}\left(N \log^2 N\right)$ for construcing suffix array and $\mathcal{O}(n)$ for construcing LCP array
Status: Tested on judge.yosupo.jp

981801, 54 lines

```
struct SuffixArray {
    string str;
    vector<int> sa, lcp, order, rank;
    vector<ll> weight;
    int n;
```

```
SuffixArray(string _str = "") {
    str = _str, n = str.size();
    sa.assign(n, 0), lcp.assign(n, 0), order.
        assign(n, 0);
    rank.assign(n, 0), weight.assign(n, 0);
    getSA(), getLCP();
}

void getSA() {
    for (int i = 0; i < n; ++i) {
        sa[i] = i;
        weight[i] = (unsigned char)str[i];
    }

    auto cmp = [&](int x, int y) {
        return weight[x] < weight[y];
    };

    sort (sa.begin(), sa.end(), cmp);

    for (int len = 1; len <= n; len <<= 1) {
        int cnt = 0;
        for (int i = 0; i < n; ++i) {
            if (i > 0 && weight[sa[i]] ==
                weight[sa[i - 1]]) order[sa[i
                ]] = order[sa[i - 1]];
            else order[sa[i]] = ++cnt;
        }
        if (cnt == n) break;
        for (int i = 0; i < n; ++i) {
            weight[i] = 1ll * order[i] * (n +
                 1) + (i + len < n ? order[i
                + len] : 0);
        }
        for (int i = 0, j = 0; i < n; i = j)
            {
            while (j < n && order[sa[j]] ==
                order[sa[i]]) ++j;
            sort (sa.begin() + i, sa.begin()
                + j, cmp);
        }
    }
    }

    void getLCP() {
        for (int i = 0; i < n; ++i) rank[sa[i]] =
             i;
        for (int i = 0, k = 0; i < n; lcp[rank[i
            ++]] = k) {
            if (rank[i] > 0) {
                int j = sa[rank[i] - 1];
                if (k > 0) --k;
                while (i + k < n && j + k < n &&
                    str[i + k] == str[j + k]) ++k
                    ;
            }
            else k = 0;
        }
    }
} T;
```

Hashing-codeforces.h

Description: Various self-explanatory methods for string hashing. Use on Codeforces, which lacks 64-bit support and where solutions can be hacked.

Status: stress-tested

<sys/time.h> cdfb3d, 52 lines

```
typedef unsigned long long ull;
static int C; // initialized below

// Arithmetic mod two primes and 2^32
    simultaneously.
// "typedef uint64_t H;" instead if Thue-Morse
    does not apply.
template<int M, class B>
struct A {
  int x; B b; A(int x=0) : x(x), b(x) {}
  A(int x, B b) : x(x), b(b) {}
  A operator+(A o){int y = x+o.x; return{y - (y>=
      M)*M, b+o.b};}
  A operator-(A o){int y = x-o.x; return{y + (y<
      0)*M, b-o.b};}
  A operator*(A o) { return {(int)(1LL*x*o.x % M)
      , b*o.b}; }
  explicit operator ull() const { return x ^ (ull
      ) b << 21; }
  bool operator==(A o) const { return (ull)*this
      == (ull)o; }
  bool operator<(A o) const { return (ull)*this <
       (ull)o; }
};
typedef A<1000000007, A<1000000009, unsigned>> H;

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(
      ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i
        -length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*
    C+c;return h;}

int main() {
  timeval tp;
  gettimeofday(&tp, 0);
  C = (int)tp.tv_usec; // (less than modulo)
```

```
  assert((ull)(H(1)*2+1-3) == 0);
  // ...
}
```

Hashing.h

Description: Self-explanatory methods for string hashing.
Status: stress-tested

f81ae8, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef unsigned long long ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};

vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0...x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$
Status: Tested on kattis:stringmatching

d4375c, 16 lines

```
vi pi(const string& s) {
  vi p(sz(s));
  rep(i,1,sz(s)) {
    int g = p[i-1];
    while (g && s[i] != s[g]) g = p[g-1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}

vi match(const string& s, const string& pat) {
  vi p = pi(pat + '\0' + s), res;
  rep(i,sz(p)-sz(s),sz(p))
    if (p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
  return res;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$
Status: Stress-tested

e7ad79, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$
Status: Stress-tested

d07a42, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

PalindromicTree.h

Description: Builds palindromic tree for a string.
Status: stress-tested

f8784d, 22 lines

```
namespace PalindromicTree {
const int N = 100000 + 5, sigma = 26;
int s[N], len[N], link[N], to[N][sigma], n, last,
    sz;
void init() {
    fill(len, len + sz, 0); fill(link, link + sz,
        0);
    for (int i = 0; i < sz; ++i) fill(to[i], to[i
        ] + sigma, 0);
    n = last = 0; s[n++] = -1; link[0] = 1; len
        [1] = -1; sz = 2;
}
int get_link(int v) {
    while (s[n - len[v] - 2] != s[n - 1]) v =
        link[v];
    return v;
}
void add_letter(int c) {
    s[n++] = c; last = get_link(last);
    if (!to[last][c]) {
        len[sz] = len[last] + 2;
        link[sz] = to[get_link(link[last])][c];
        to[last][c] = sz++;
    }
    last = to[last][c];
}
}
```

SuffixAutomaton.h

Description: Suffix Automaton, supporting multiple strings. The beginning iterator is 0. Make calls to itmove to progress iterator.
Time: $\mathcal{O}(n)$
Status: stress-tested

be95c5, 29 lines

```
struct SuffixAutomaton{
    struct Node{
        int len, link = -1, nxt[26];
        Node(int len): len(len) {fill(nxt, nxt +
            26, -1);}
        Node(int len, int link, int _nxt[26]):
        len(len), link(link) {memcpy(nxt, _nxt,
            sizeof nxt);}
    };
    vector<Node> g = {0};
    int ptrans(int p, char c, int nxt) {
        for (int q = g[p].nxt[c]; p != -1 && g[p
            ].nxt[c] == q;
            p = g[p].link) g[p].nxt[c] = nxt;
        return p;
    }
    int fbuild(int p, char c) {
        if (p == -1) return 0;
        int q = g[p].nxt[c];
        if (g[p].len + 1 == g[q].len) return q;
        int qc = g.size();
        g.emplace_back(g[p].len+1, g[q].link, g[q
            ].nxt);
        ptrans(p, c, qc); return g[q].link = qc;
    }
    int addc(int p, char c) { // Progress
        iterator; c is in [0, 26).
        if (g[p].nxt[c] != -1) return fbuild(p, c
            );
        int nc = g.size();
        g.emplace_back(g[p].len+1);
        int r = fbuild(ptrans(p, c, nc), c);
        g[nc].link = r; return nc;
    }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).
Time: $\mathcal{O}(26N)$
Status: stress-tested a bit

aae0b8, 50 lines

```
struct SuffixTree {
enum { N = 200010, ALPHA = 26 }; // N ~ 2*
    maxlen+10
int toi(char c) { return c - 'a'; }
string a; // v = cur node, q = cur position
int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m
    =2;

void ukkadd(int i, int c) { suff:
  if (r[v]<=q) {
    if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
      p[m++]=v; v=s[v]; q=r[v];  goto suff; }
    v=t[v][c]; q=l[v];
  }
  if (q==-1 || c==toi(a[q])) q++; else {
    l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
    p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v
        ;
    l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
    v=s[p[m]];  q=l[m];
    while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v
        ]-l[v]; }
    if (q==r[m])  s[m]=v;  else s[m]=m+2;
    q=r[v]-(q-r[m]);  m+=2;  goto suff;
  }
}

SuffixTree(string a) : a(a) {
  fill(r,r+N,sz(a));
  memset(s, 0, sizeof s);
  memset(t, -1, sizeof t);
  fill(t[1],t[1]+ALPHA,0);
  s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p
      [0] = p[1] = 0;
  rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
}

// example: find longest common substring (uses
    ALPHA = 28)
pii best;
int lcs(int node, int i1, int i2, int olen) {
  if (l[node] <= i1 && i1 < r[node]) return 1;
  if (l[node] <= i2 && i2 < r[node]) return 2;
```

```
    int mask = 0, len = node ? olen + (r[node] -
        l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
      best = max(best, {len, r[node] - len});
    return mask;
  }
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char
        )('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
};
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$
Status: stress-tested

ee09e2, 12 lines

```
vi Z(const string& S) {
  vi z(sz(S));
  int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z
        [i]])
      z[i]++;
    if (i + z[i] > r)
      l = i, r = i + z[i];
  }
  return z;
}
```

Various (10)

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.
Status: proven correct, stress-tested

62d9a9, 7 lines

```
struct FastMod {
  uint64_t b, m;
  FastMod(uint64_t b) : b(b), m(-1ULL / b) {}
  uint64_t reduce(uint64_t a) { // a % b + (0 or
      b)
    return a - (uint64_t)((__uint128_t(m) * a) >>
        64) * b;
  }
};
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.
Status: tested

745db2, 8 lines

```
// Either globally or in a single class:
static char buf[450 << 20];
void* operator new(size_t s) {
  static size_t i = sizeof buf;
  assert(s < i);
  return (void*)&buf[i -= s];
}
void operator delete(void*) {}
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: `vector<vector<int, small<int>>> ed(N);`
Status: tested

bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
  typedef T value_type;
  small() {}
  template<class U> small(const U&) {}
  T* allocate(size_t n) {
    buf_ind -= n * sizeof(T);
    buf_ind &= 0 - alignof(T);
    return (T*)(buf + buf_ind);
  }
  void deallocate(T*, size_t) {}
};
```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.
Usage: `constantIntervals(0, sz(v), [&](int x){return v[x];}, [&](int lo, int hi, T val){...});`
Time: $\mathcal{O}\left(k \log \frac{n}{k}\right)$
Status: tested

753a4c, 19 lines

```
template<class F, class G, class T>
void rec(int from, int to, F& f, G& g, int& i, T&
     p, T q) {
  if (p == q) return;
  if (from == to) {
    g(i, to, p);
    i = to; p = q;
  } else {
    int mid = (from + to) >> 1;
    rec(from, mid, f, g, i, p, f(mid));
    rec(mid+1, to, f, g, i, p, q);
  }
}
template<class F, class G>
void constantIntervals(int from, int to, F f, G g
    ) {
  if (to <= from) return;
  int i = from; auto p = f(i), q = f(to-1);
  rec(from, to-1, f, g, i, p, q);
  g(i, to, q);
}
```

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \le k < hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}\left((N + (hi - lo)) \log N\right)$
Status: tested on http://codeforces.com/contest/321/problem/E

d38d2b, 18 lines

```
struct DP { // Modify at will:
  int lo(int ind) { return 0; }
  int hi(int ind) { return ind; }
```

```
  ll f(int ind, int k) { return dp[ind][k]; }
  void store(int ind, int k, ll v) { res[ind] =
      pii(k, v); }

  void rec(int L, int R, int LO, int HI) {
    if (L >= R) return;
    int mid = (L + R) >> 1;
    pair<ll, int> best(LLONG_MAX, LO);
    rep(k, max(LO,lo(mid)), min(HI,hi(mid)))
      best = min(best, make_pair(f(mid, k), k));
    store(mid, best.second, best.first);
    rec(L, mid, LO, best.second+1);
    rec(mid+1, R, best.second, HI);
  }
  void solve(int L, int R) { rec(L, R, INT_MIN,
      INT_MAX); }
};
```

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.
Usage: `./a.out < input.txt`
Time: About 5x as fast as cin/scanf.
Status: tested on SPOJ INTEST, unit tested

7b3c70, 16 lines

```
inline char gc() { // like getchar()
  static char buf[1 << 16];
  static size_t bc, be;
  if (bc >= be) {
    buf[0] = 0, bc = 0;
    be = fread(buf, 1, sizeof(buf), stdin);
  }
  return buf[bc++]; // returns 0 on EOF
}
int readInt() {
  int a, c;
  while ((a = gc()) < 40);
  if (a == '-') return -readInt();
  while ((c = gc()) >= 48) a = a * 10 + c - 480;
  return a - 48;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t, computes the maximum S <= t such that S is the sum of some subset of the weights.
Time: $\mathcal{O}(N \max(w_i))$
Status: Tested on kattis:eavesdropperevasion, stress-tested

b20ccc, 16 lines

```
int knapsack(vi w, int t) {
  int a = 0, b = 0, x;
  while (b < sz(w) && a + w[b] <= t) a += w[b++];
  if (b == sz(w)) return a;
  int m = *max_element(all(w));
  vi u, v(2*m, -1);
  v[a+m-t] = b;
  rep(i,b,sz(w)) {
    u = v;
    rep(x,0,m) v[x+w[i]] = max(v[x+w[i]], u[x]);
    for (x = 2*m; --x > m;) rep(j, max(0,u[x]), v
        [x])
      v[x-w[j]] = max(v[x-w[j]], j);
  }
  for (a = t; v[a+m-t] < 0; a--) ;
  return a;
}
```

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
Time: $\mathcal{O}(\log N)$
Status: stress-tested

edce47, 22 lines

```
set<pii>::iterator addInterval(set<pii>& is, int
    L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}
void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add `|| R.empty()`. Returns empty set on failure (or if G is empty).
Time: $\mathcal{O}(N \log N)$
Status: Tested on kattis:intervalcover

9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I) {
  vi S(sz(I)), R;
  iota(all(S), 0);
  sort(all(S), [&](int a, int b) { return I[a] <
      I[b]; });
  T cur = G.first;
  int at = 0;
  while (cur < G.second) { // (A)
    pair<T, int> mx = make_pair(cur, -1);
    while (at < sz(I) && I[S[at]].first <= cur) {
      mx = max(mx, make_pair(I[S[at]].second, S[
          at]));
      at++;
    }
    if (mx.second == -1) return {};
    cur = mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$, where the (minimal) optimal k increases with both i and j, one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b,c) \leq f(a,d)$ and $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}\left(N^2\right)$

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N \log N)$

Status: Tested on kattis:longincsubseq, stress-tested

2932a0, 17 lines

```
template<class I> vi lis(const vector<I>& S) {
  if (S.empty()) return {};
  vi prev(sz(S));
  typedef pair<I, int> p;
  vector<p> res;
  rep(i,0,sz(S)) {
    // change 0 -> i for longest non-decreasing
        subsequence
    auto it = lower_bound(all(res), p{S[i], 0});
    if (it == res.end()) res.emplace_back(), it =
         res.end()-1;
    *it = {S[i], i};
    prev[i] = it == res.begin() ? 0 : (it-1)->
        second;
  }
  int L = sz(res), cur = res.back().second;
  vi ans(L);
  while (L--) ans[L] = cur, cur = prev[cur];
  return ans;
}
```

SIMD.h

Description: Cheat sheet of SSE/AVX intrinsics, for doing arithmetic on several numbers at once. Can provide a constant factor improvement of about 4, orthogonal to loop unrolling. Operations follow the pattern "_mm(256)?_name_(si(128|256)|epi(8|16|32|64)|pd|ps)". Not all are described here; grep for _mm_ in /usr/lib/gcc/*/4.9/include/ for more. If AVX is unsupported, try 128-bit operations, "emmintrin.h" and #define __SSE__ and __MMX__ before including it. For aligned memory use _mm_malloc(size, 32) or int buf[N] alignas(32), but prefer loadu/storeu.

c9ac08, 43 lines

```
#pragma GCC target ("avx2") // or sse4.1
#include "immintrin.h"

typedef __m256i mi;
#define L(x) _mm256_loadu_si256((mi*)&(x))

// High-level/specific methods:
// load(u)?_si256, store(u)?_si256, setzero_si256
    , _mm_malloc
// blendv_(epi8|ps|pd) (z?y:x), movemask_epi8 (
    hibits of bytes)
// i32gather_epi32(addr, x, 4): map addr[] over
    32-b parts of x
// sad_epu8: sum of absolute differences of u8,
    outputs 4xi64
// maddubs_epi16: dot product of unsigned i7's,
    outputs 16xi15
// madd_epi16: dot product of signed i16's,
    outputs 8xi32
// extractf128_si256(, i) (256->128),
    cvtsi128_si32 (128->lo32)
// permute2f128_si256(x,x,1) swaps 128-bit lanes
// shuffle_epi32(x, 3*64+2*16+1*4+0) == x for
    each lane
// shuffle_epi8(x, y) takes a vector instead of
    an imm

// Methods that work with most data types (append
    e.g. _epi32):
// set1, blend (i8?x:y), add, adds (sat.), mullo,
    sub, and/or,
// andnot, abs, min, max, sign(1,x), cmp(gt|eq),
    unpack(lo|hi)

int sumi32(mi m) { union {int v[8]; mi m;} u; u.m
    = m;
  int ret = 0; rep(i,0,8) ret += u.v[i]; return
     ret; }
mi zero() { return _mm256_setzero_si256(); }
mi one() { return _mm256_set1_epi32(-1); }
bool all_zero(mi m) { return _mm256_testz_si256(m
    , m); }
bool all_one(mi m) { return _mm256_testc_si256(m,
    one()); }

ll example_filteredDotProduct(int n, short* a,
    short* b) {
  int i = 0; ll r = 0;
  mi zero = _mm256_setzero_si256(), acc = zero;
  while (i + 16 <= n) {
    mi va = L(a[i]), vb = L(b[i]); i += 16;
    va = _mm256_and_si256(_mm256_cmpgt_epi16(vb,
        va), va);
    mi vp = _mm256_madd_epi16(va, vb);
    acc = _mm256_add_epi64(_mm256_unpacklo_epi32(
        vp, zero),
      _mm256_add_epi64(acc, _mm256_unpackhi_epi32
          (vp, zero)));
  }
  union {ll v[4]; mi m;} u; u.m = acc; rep(i,0,4)
      r += u.v[i];
  for (;i<n;++i) if (a[i] < b[i]) r += a[i]*b[i];
      // <- equiv
  return r;
}
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

Status: tested

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
  unsigned ind;
```

```
  ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf
      ) : 0) {
    assert(ind < sizeof buf);
  }
  T& operator*() const { return *(T*)(buf + ind);
       }
  T* operator->() const { return &**this; }
  T& operator[](int a) const { return (&**this)[a
      ]; }
  explicit operator bool() const { return ind; }
};
```

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$
Status: tested

9155b4, 11 lines

```
template<class F>
int ternSearch(int a, int b, F f) {
  assert(a <= b);
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  }
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
}
```

Unrolling.h

520e76, 5 lines

```
#define F {...; ++i;}
int i = from;
while (i&3 && i < to) F // for alignment, if
    needed
while (i + 4 <= to) { F F F F }
while (i < to) F
```

Techniques (A)

techniques.txt

159 lines

```
Recursion
Divide and conquer
  Finding interesting points in N log N
Algorithm analysis
  Master theorem
  Amortized time complexity
Greedy algorithm
  Scheduling
  Max contiguous subvector sum
  Invariants
  Huffman encoding
Graph theory
  Dynamic graphs (extra book-keeping)
  Breadth first search
  Depth first search
  * Normal trees / DFS trees
  Dijkstra's algorithm
  MST: Prim's algorithm
  Bellman-Ford
  Konig's theorem and vertex cover
  Min-cost max flow
  Lovasz toggle
  Matrix tree theorem
  Maximal matching, general graphs
  Hopcroft-Karp
  Hall's marriage theorem
  Graphical sequences
  Floyd-Warshall
  Euler cycles
  Flow networks
  * Augmenting paths
  * Edmonds-Karp
  Bipartite matching
  Min. path cover
  Topological sorting
  Strongly connected components
  2-SAT
  Cut vertices, cut-edges and biconnected
      components
  Edge coloring
  * Trees
  Vertex coloring
  * Bipartite graphs (=> trees)
  * 3^n (special case of set cover)
  Diameter and centroid
  K'th shortest path
  Shortest cycle
Dynamic programming
  Knapsack
  Coin change
  Longest common subsequence
  Longest increasing subsequence
  Number of paths in a dag
  Shortest path in a dag
  Dynprog over intervals
  Dynprog over subsets
  Dynprog over probabilities
  Dynprog over trees
  3^n set cover
  Divide and conquer
  Knuth optimization
  Convex hull optimizations
  RMQ (sparse table a.k.a 2^k-jumps)
  Bitonic cycle
  Log partitioning (loop over most restricted)
Combinatorics
  Computation of binomial coefficients
  Pigeon-hole principle
  Inclusion/exclusion
  Catalan number
  Pick's theorem
Number theory
  Integer parts
  Divisibility
  Euclidean algorithm
  Modular arithmetic
  * Modular multiplication
  * Modular inverses
  * Modular exponentiation by squaring
  Chinese remainder theorem
  Fermat's little theorem
  Euler's theorem
  Phi function
  Frobenius number
  Quadratic reciprocity
  Pollard-Rho
  Miller-Rabin
  Hensel lifting
  Vieta root jumping
Game theory
  Combinatorial games
  Game trees
  Mini-max
  Nim
  Games on graphs
  Games on graphs with loops
  Grundy numbers
  Bipartite games without repetition
  General games without repetition
  Alpha-beta pruning
Probability theory
Optimization
  Binary search
  Ternary search
  Unimodality and convex functions
  Binary search on derivative
Numerical methods
  Numeric integration
  Newton's method
  Root-finding with binary/ternary search
  Golden section search
Matrices
  Gaussian elimination
  Exponentiation by squaring
Sorting
  Radix sort
Geometry
  Coordinates and vectors
  * Cross product
  * Scalar product
  Convex hull
  Polygon cut
  Closest pair
  Coordinate-compression
  Quadtrees
```

```
  KD-trees
  All segment-segment intersection
Sweeping
  Discretization (convert to events and sweep)
  Angle sweeping
  Line sweeping
  Discrete second derivatives
Strings
  Longest common substring
  Palindrome subsequences
  Knuth-Morris-Pratt
  Tries
  Rolling polynomial hashes
  Suffix array
  Suffix tree
  Aho-Corasick
  Manacher's algorithm
  Letter position lists
Combinatorial search
  Meet in the middle
  Brute-force with pruning
  Best-first (A*)
  Bidirectional search
  Iterative deepening DFS / A*
Data structures
  LCA (2^k-jumps in trees in general)
  Pull/push-technique on trees
  Heavy-light decomposition
  Centroid decomposition
  Lazy propagation
  Self-balancing trees
  Convex hull trick (wcipeg.com/wiki/
      Convex_hull_trick)
  Monotone queues / monotone stacks / sliding
      queues
  Sliding queue using 2 stacks
  Persistent segment tree
```

Test session (B)

B.1 To test during test session

- Keyboard layouts
- bashrc
- vimrc
- template.cpp
- Java
- Python
- Printing
- Sending clarifications
- Documentation
- Submit script
- Whitespace/case insensitivity in output
- Return values from main
- Printing to stderr
- Source code limit
- Is it possible to submit and read from non-source files?
- SIMD support on machine resp. judge
- How long do array accesses take on machine resp. judge?
- How long do small operations take on machine resp. judge?
- Does `clock()` work?
- All judge errors:
  - Accepted
  - WA
  - Presentation Error
  - RTE (what results in RTE?)
  - TLE
  - Memory limit, both stack and heap
  - Output limit
  - Illegal function
  - Compile error
  - Compile time limit exceeded
  - Compile memory limit exceeded
  - Too late
- Listing all available binaries
  `echo $PATH | tr ':' ' ' | xargs ls | grep -v / | sort | uniq | tr '\n' ' ' > path.txt`